关于微网与外部电网电力调控策略研究

摘 要

随着分布式电源渗透率不断提高，微网需与外部电网灵（以下简称外网）活互动，通过合理的电力调控保障供电可靠性与运行经济性。本文主要研究了微网与外网的电力调控策略问题，综合建立了混合整数线性规划模型（MILP），并基于“场景随机优化＋滚动调整＋反馈控制”的方法进行优化与求解。

针对问题一，主要解决在电价、负载相同且光伏预测已知的条件下单天计划购电量和储能充放电安排优化问题，首先读取附件 1 的电价、负载与光伏预测功率数据进行预处理工作，发现无缺失、非数值或负值。然后采用线性规划方法，以全天购电费用最小为目标，以电能供需平衡、储能容量及充放电功率边界为约束，充电模式为 0-1 变量，建立了混合整数线性规划模型，并利用两阶段求解方法进行了求解，得出全天购电量为 59482.70kWh，全天购电费为 35126.95 元的结果。

针对问题二，以“不可预知的供需变化”与“0:00 必须提前购电”之间的矛盾为核心，构建 0 时求解日前场景线性规划模型，充分考虑信息假设和预测场景的可靠性，利用日前模型和日内反馈算法求解，确定总实际购电费用为 13,672,895.40 元，经过基建与公平性检验结果在给定条件下可稳定复现。

针对问题三，本研究引入多时点光伏预报更新和日内购电调整机制，建立了“日前两阶段随机优化＋日内滚动调整＋储能反馈”模型。采用历史联合场景刻画不确定性，先以 0:00 两阶段随机规划确定初始购电计划，再在 6:00、12:00 和 18:00 滚动优化剩余时段，并通过储能价值函数逐 10 分钟反馈调整充放电，得到 2025 年 2 月 1 日至 12 月 31 日 334 天实际总购电费用为 13,575,585.07 元的结果。除此之外，模型形成“少承诺、后补购”的稳健结构，并与完美预知下界的差距缩小到约 9.91%。

针对问题四，本研究以附件 4 真实电价重算问题二与问题三.模型以购电、增购、违约和应急费用最小为目标，利用线性规划消除无效充放循环，并通过 100 kWh 储能价值反馈求解。结果表明，在价格波动的条件下，问题二全年费用 **14,410,716.10** 元，应急电量 **79,181.70 kWh**；问题三全年费用 **14,329,837.01 元，**应急电量降至 27,130.03 kWh。模型中电价、预报更新时间、各类结算系数以及储能容量、功率和效率均可作为参数调整，因此既能处理固定电价，也能推广到实时电价、不同更新频率和不同交易规则的情形，具有较强的普适性。

关键词：微网与外网；MILP；历史残差场景；滚动模型预测控制

1、问题重述

1.1 问题背景

随着社会水平和科技发展的进步，分布式发电规模不仅持续扩大，而且呈现多元化、智能化的发展趋势。然而，分布式电源直接接入电网会给电力系统的管理带来困难，并可能引发电能质量恶化以及电网安全性与稳定性下降等问题。[1] 微网是一种将分布式电源、储能系统、能量转换装置、监控保护装置及负荷等整合在一起的小型发配用电一体化系统，具备自我控制与管理能力，既可灵活并网运行，也能转入孤岛模式独立运行，在节约能源、降低损耗以及提升供电可靠性等方面展现出巨大的应用潜力。[2]

1.2 问题提出

以微网提供的电力能满足小区的负载且最大程度节省微网的购电费用为目的，本研究分析用小区负载、光伏发电量和储能设备的当前储电量等数据，得到精准的微网购电量和储能设备的当前储电量。结合附件中的具体数据，解决如下问题：

问题一：假设每天的电价和小区负载相同，并已知光伏发电功率的日内预测结果，需要在每天 0 时制定购电的数学模型。模型应以全天购电费用最低为目标，保证任何时段微网提供的电能不低于小区负荷，并要求储能设备在当天 0 时和 24 时的电量相同。依据附件 1 及附录 1 求解计划购电量和储能充放电安排，并按照规定格式整理表 1 和表 2 中的购电量、充电量和放电量等数据。

问题二：假设每天的店家相同，而小区负荷和光伏实际发电功率每日不同。如果微网提供的电能低于负载，需要向外部电网紧急购电，其价格为交易时刻电价的 5 倍；除紧急购电外，其他时间段的购电费用仍按照计划购电量计算。全面考虑上述因素，建立每天 0 时制定当天计划购电策略的模型，根据附件 1 和附件 2 的数据进行求解，并分析指定日期的计划购电、储能运行和紧急购电情况，填写文件 result2.xlsx。

问题三：立足于问题二，在每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率的条件下，微网可以先根据 0 点的预报制定初始购电计划，也可在后续预报时刻调整计划策略。对于计划购电量超出调整购电量的部分按照交易时刻电价的 50%计违约电价，对于计划购电量低于调整购电量的部分按照交易时刻电价的 1.5 倍计价。总购电费由计划购电费用、紧急购电费用和调整购电量的相关费用组成。综合分析上述条件，建立滚动调整模型，确定各预报时刻的计划与调整策略，填写文件 result3.xlsx，并进一步分析参考其他时刻的预报调整购电策略的必要性。

问题四：实际上外部电网价格具有波动性。在问题二和问题三的基础上，引入附件 4 提供的实时电价数据，重新建立微网购电与储能调度模型，并将完整购电策略分别保存到文件 result4-2.xlsx 和 result4-3.xlsx 中。

2、问题分析

针对以上问题，本文采用了以下整体思路框架进行系统性研究和分析：

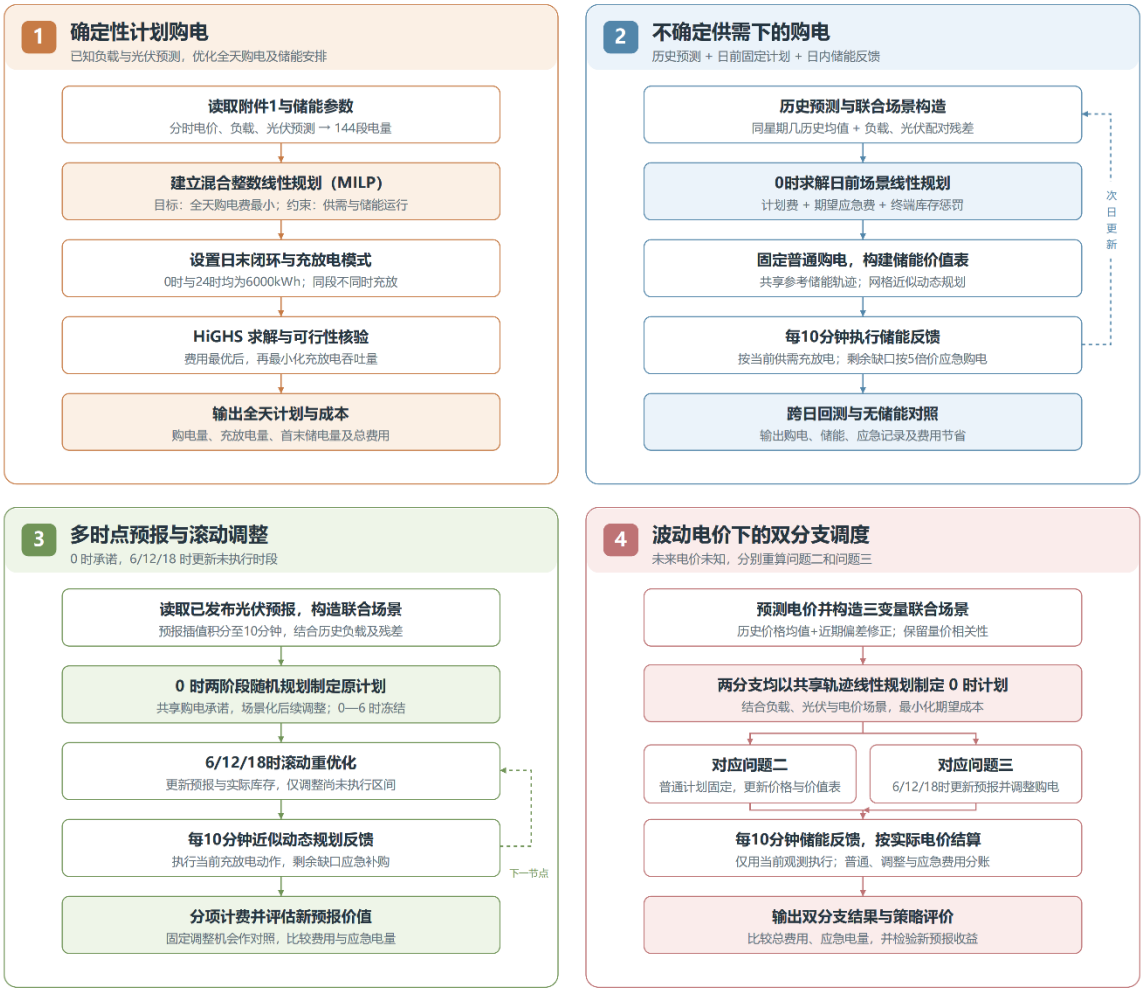


图 1 问题的总分析

3、模型假设

1. 将一天划分为 144 个 10 分钟时段，各时段内功率和电价不变，功率乘以 1/6 转换为电量。

2. 所有决策仅使用当时已获得的信息，未来的负载、光伏功率和电价只用于事后结算与验证。

3. 微网不向外部电网售电，光伏和计划购电的富余部分允许不利用，记为未利用电量。

4. 外网可满足购电和紧急购电需求，不额外设置线路功率上限，紧急购电价格为当期电价的 5 倍。

5.储能充放电效率均取 0.9，储电量保持在 1200 至 10800 kWh，功率不超过 5000 kW，且禁止同时充放电。

6.负载按最近 8 个相同星期几的历史均值预测，负载和光伏残差按同一历史日联合生成最多 7 个等权场景。

7.原计划购电量全额计费；增购按 1.5 倍电价、减购按 0.5 倍电价收取违约费，多次调整按最终净差结算一次。

8.问题一要求日初、日末储电量均为 6000 kWh，问题二至问题四仅保持储能状态跨日连续。

4、符号说明

表 1 本文的符号说明

符号	说明	单位
L_t	小区负载功率	元 / kW
P_t	光伏功率（预测或实际）	kW
π_t	电价	元 / kW
x_t	计划购电功率（问题三记x_t^{plan}）	kW
x_t^{adj}	调整购电功率	kW
y_t	紧急购电功率	kW
g_t	弃光功率（消减的光伏）	kW
c_t, d_t	储能充电、放电功率	
S_t	储能电量（时段t末）	kW
u_t, v_t	少购偏差、超购偏差（问题 3）	kW
η	充放电效率（=0.9）	\
$P_{\max}, E_{\max}$	最大充放电功率、最大容量	kW / kWh
$S_{\min}, S_{\max}, S_{\text{init}}$	电量下届、上届、初始电量	kWh

5、模型建立与求解

5.1 问题一模型的建立与求解

问题一要求在给定电价、小区负载和光伏发电预测功率的条件下，确定储能系统的充放电策略以及微网购电量，使全天购电费用尽可能低。

考虑到数据以 10 分钟为间隔记录，本文将一天划分为 144 个时段，设时段长度为

$$\Delta t = \frac{1}{6}\text{h}$$

在每个时段内，将负载功率和光伏发电功率转换为对应电量，并以购电量、充电量和放电量作为主要决策变量。储能系统能够在低电价或光伏富余时段储存能量，并在高购电成本时段释放能量。除此之外，时段的充电模式本质上是“充与不充”的 0-1 选择，因此问题可以转化为一个带储能状态约束的混合整数线性规划模型。

5.1.1 数据预处理

读取附件 1 的电价、负载与光伏预测功率数据核对是否有 144 条记录，有没有缺失、非数值或负值。将“00:10”对应到 00:00—00:10，依次覆盖全天。除此之外，附件给的是功率，单位为 kW；模型需要每 10 分钟的电量，单位为 kWh。所以需要功率转换为电量的单位换算，即$E_t = P_t \times \frac{10}{60}$ (kWh)。

同时，需要注意的是时间段是 1 到 144，共 144 段；储电状态是 0 到 144，共 145 个时刻。购电量、充电量、放电量等，每段一个数，都是 144 个数。储电量包含最开始的状态和每段结束后的状态，有 145 个数。

5.1.2 MILP 模型的建立

1.目标函数

以全天购电费用最小为目标：

$$\min J_1 = \sum_{t=1}^{144} p_t G_t \tag{1}$$

其中，J_1、P_t和G_t分别表示全天购电费、第 t 时段购电价格和购电量。$P_t G_t$表示某一时段的购电费用，对 144 个时段求和后得到全体总购电费用J。模型通过优化G_t及储能充放电量，使全天总购电费用最少。

2.电能供需平衡约束

每个时段中电能都应维持供需平衡，即电网购电、光伏发电和储能放电的总和等于负载、储能充电及未利用电量，因此有

$$G_t + V_t + b_t = L_t + c_t + w_t, t = 1,2,\dots,144 \tag{2}$$

其中，V_t表示第 t 时段光伏发电量，b_t表示第 t 时段放电量，L_t表示第 t 时段负载电量，c_t表示第 t 时段充电量，w_t表示第 t 时段未利用电量。

式（2）左侧为系统的可用电能，右侧为负载消耗电能、储能充电能和未利用电量。若光伏发电量超过负载和储能可吸收的电量，超出部分由w_t表示。

3.储能状态递推约束

储能系统的电量部分来自前一时段的继承：当前储电量=上一时段储电量+本时段充电增量-本时段放电消耗量，即：

$$E_t = E_{t-1} + \eta_c c_t - \frac{b_t}{\eta_d}, t = 1,2,\ldots,144 \tag{3}$$

E_t：第 t 时段末储能内部电量；

E_{t-1}：上一时段结束时的剩余电量；

η_c：充电效率（=0.9）；

η_d：放电效率（=0.9）。

式（3）描述了储能系统在相邻时段间的电量递推关系，E_{t-1}为时段初电池内部电量，

$\eta_c c_t$为充电过程中扣除效率损耗后实际输入电池的电量，$\frac{b_t}{\eta_d}$为满足母线侧放电需求所需消耗的电池内部电量。该式充分反映储能系统“跨时段转移电能”的作用。式中所采用的充电效率和放电效率均为 0.9，对应往返效率为 0.9×0.9=0.81。

4.储能容量与边界约束

为保证储能系统安全运行，储电量应始终处于允许区间内：

$$1200 \le E_t \le 10800, t = 1,2,\ldots,144 \tag{4}$$

其中，E_t为第 t 时段末储能内部电量。同时，0 时和 24 时储能状态应满足：

$$E_0 = 6000, E_{144} = 6000 \tag{5}$$

其中，E_0表示 0 时的储电量，E_{144}此表示 24 时的储电量。该式保证了储能系统在一天结束时恢复到初始状态，保持持续运行的能力避免电池加速老化。

5.充放电功率与互斥约束

每个 10 分钟时段的最大充放电电量为 5000/6kWh，引入$z_t \in \{0,1\}$ ，建立约束：

$$0 \le c_t \le \frac{5000}{6} z_t \tag{6}$$

$$0 \le b_t \le \frac{5000}{6}(1 - z_t) \tag{7}$$

其中z_t表示充电模式 0-1 变量，当z_t=1 时，储能只能充电，当z_t=0 时，储能只能放电。此外，充电能量应由购电和光伏共同提供：

$$c_t \le G_t + V_t \tag{8}$$

所有决策变量满足非负约束：

$$G_t, c_t, b_t, w_t \ge 0 \tag{9}$$

式（1）—式（9）构成问题一的混合整数线性规划模型。

5.1.3 两阶段求解方法

模型中包含 144 个 0-1 变量，本文采用 SciPy 调用 HiGHS 求解器进行混合整数线性规划求解，求解过程分为两个阶段。

1.第一阶段最小化购电费用：

$$\min J_1 = \sum_{t=1}^{144} p_t G_t$$

记最优费用为J_1^*。

2.第二阶段：基于最小总充放电量的最优解筛选

为使结果稳定、便于复现，第二阶段在允许费用略有增加的前提下最小化总充放电吞吐量：

$$\min J_{12} = \sum_{t=1}^{144} (c_t + b_t) \tag{10}$$

并增加约束：

$$J_1 \le J_1^* + 10^{-7} \tag{11}$$

该步骤可以在不显著影响经济性的情况下减少不必要的储能循环，在费用增加不超过10^{-7}元的容差内最小化总充放吞吐量，以选择一组稳定可复现的最优计划。

5.1.4 求解结果与调度策略分析

经上述求解，该小区全天购电费用为 **35126.948589** 元，按分显示为 35126.95 元，0:00 和 24:00 储电量均为 6000 kWh。一个 4 小时汇总段可以同时出现充、放电量。这表示该段中不同的 10 分钟时段分别充电或放电，不是同一时刻同时充放。

全天各时段计划购电策略结果写入文件 result1.xlsx。微网在指定时间段的购电量及全天的购电量和购电费见表 1，储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量见表 2。

如下图，图(a)为全天负载与光伏预测功率，午间光伏出力显著增加；(b)表明储电量始终处于 1200—10800 kWh 允许范围，并在 24:00 恢复至 6000 kWh；(c)表明储能主要在低电价或光伏富余时段充电、在高购电成本时段放电，且同一时段充放电互斥；(d)显示无储能方案全天购电费用为 48052.05 元，优化调度后降至 35126.95 元，节省 12925.10 元，降幅约 26.90%。结果表明，问题一所建立模型能够通过储能跨时段转移电能，在满足功率平衡和储能运行约束的同时显著降低全天购电费用。

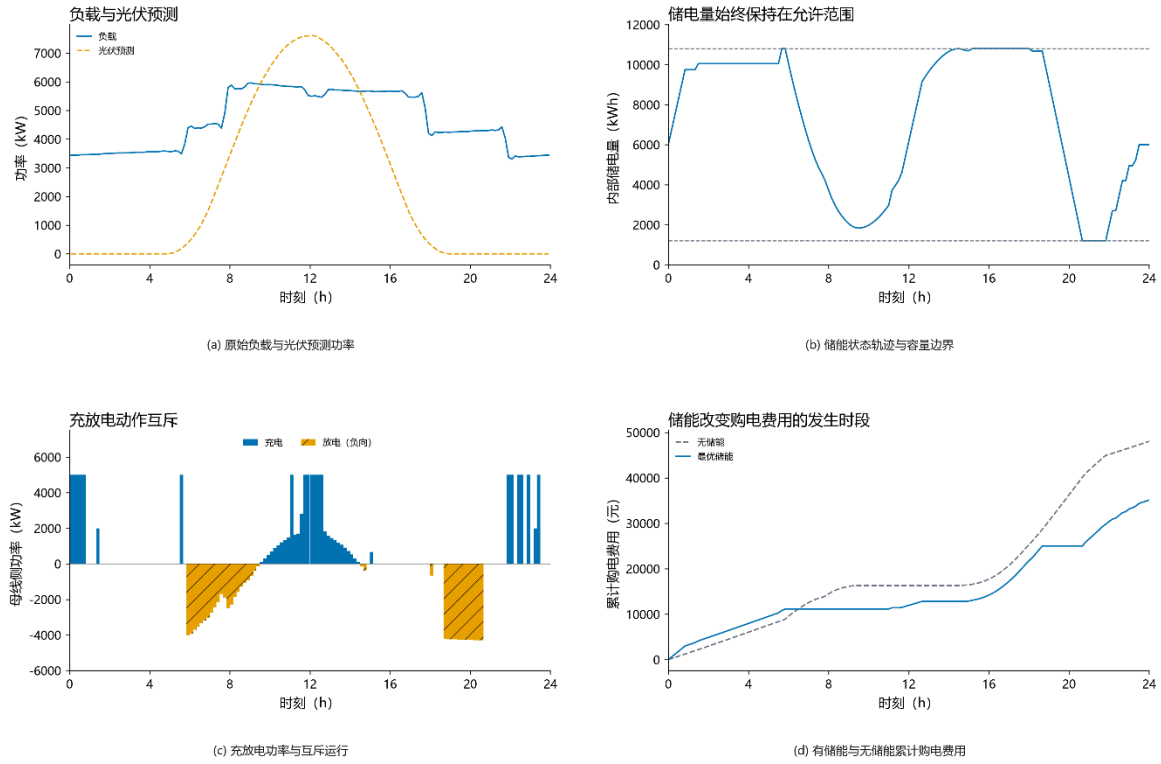


图 2 问题一储能调度结果及购电费用对比

表 2 列出了题目指定核算时段的购电量。其中，三个指定时刻呈现出鲜明的电价响应：结合图 1 可知，10:00 与 14:00 处于光伏出力主平台期。

表 2 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量（kWh）
10:00 - 10:10	0.000000
12:00 - 12:10	480.412450
14:00 - 14:10	0.000000
16:00 - 16:10	445.431650
18:00 - 18:10	531.894050
20:00 - 20:10	0.000000
全天购电量	**59482.698998**

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量（kWh）	放电量（kWh）
00:00–04:00	4500.000000	0.000000
04:00–08:00	833.333333	6365.841199
08:00–12:00	4787.964287	1702.996983
12:00–16:00	5286.035177	91.101383
16:00–20:00	0.000000	5780.131883
20:00–24:00	5333.333333	2859.868117
全天	**20740.666130**	**16799.939566**

5.1.5 模型汇总

$$
\begin{cases}
\Delta = \frac{1}{6}h; \\
\min J_1 = \sum_{t=1}^{144} p_t\, G_t; \\
G_t + V_t + b_t = L_t + c_t + w_t, t = 1,2,\dots,144; \\
E_t = E_{t-1} + \eta_c c_t - \frac{b_t}{\eta_d}, t = 1,2,\dots,144; \\
1200 \le E_t \le 10800, t = 1,2,\dots,144; \\
E_0 = 6000, E_{144} = 6000; \\
0 \le c_t \le \frac{5000}{6} z_t;\ \ 0 \le b_t \le \frac{5000}{6}(1 - z_t); \\
c_t \le G_t + V_t; \\
J_1 \le J_1^* + 10^{-7}; \\
\min J_{12} = \sum_{t=1}^{144}(c_t + b_t);
\end{cases}
$$

5.2 问题二模型的建立与求解

5.2.1 建模思路

问题二中每天的电价结构相同，但小区负载和光伏发电功率逐日、逐时段变化。若微网提供的电能低于负载，则需要按交易时刻电价的 5 倍进行紧急购电。因此，问题二需要在每天 0 点制定当天普通购电计划，同时协调储能系统充放电，以降低计划购电费用和紧急购电费风险。

由于题目要每天 00:00 制定当天购电计划，本研究采用历史信息驱动的非预知决策方式：每天 0:00 只利用之前完整日的负载和光伏数据预测未来变化，确定普通计划购

电量；进入当天后，普通购电量保持不变，微网根据当前实际负载、实际光伏和储能状态调整充放电量。若仍然无法满足负载，则进行紧急购电，并将完整的购电策略保存到文件 result2.xlsx 中。

将一天划分为$T = 144$个 10 分钟时段，即$\Delta t = \frac{1}{6}\text{h}$，设第$t$个时段的电价为$p_t$，负载功率和光伏功率分别为$P_{d,t}^{L}$和$P_{d,t}^{V}$，则对应的负载电量和光伏电量为

$$L_{d,t} = \Delta t P_{d,t}^{L} = \frac{P_{d,t}^{L}}{6}, V_{d,t} = \Delta t P_{d,t}^{V} = \frac{P_{d,t}^{V}}{6}$$

其中，$L_{d,t}$表示负载电量，$V_{d,t}$表示光伏电量，电量单位均为 kWh，电价单位为元/kWh。

5.2.2 基于同一星期几的历史预测

1.预测分组方式

预测第d日的负载与光伏时，取此前最近 8 个相同星期几的完整日期，对每天分别匹配历史上相同星期几的数据，组成历史样本集合H_d，从而避免未来信息的泄漏。目标日第t时段的负载和光伏基准预测为：

$$\hat{L}_{d,t} = \frac{1}{|H_d|}\sum_{i \in H_d} L_{i,t}, \hat{V}_{d,t} = \frac{1}{|H_d|}\sum_{i \in H_d} V_{i,t} \tag{12}$$

其中，$\hat{L}_{d,t}$表示时段负载电量平均值（预测值），$\hat{V}_{d,t}$表示时段光伏电量平均值（预测值），H_d表示与目标日具有相同日期类型的最近历史日期集合，当满足同类条件的日期少于两天时，改为选取历史窗口中的所有日期。

2.预测残差

为了避免干扰负载和光伏之间的相关关系，本文不以单时段方式重新抽样，而是完整保存每个历史日的完整预测残差对：

$$r_{i,t}^{L} = L_{i,t} - \hat{L}_{i,t}, r_{i,t}^{V} = V_{i,t} - \hat{V}_{i,t} \tag{13}$$

其中，i：某个历史日期；t：当天的第t个 10 分钟时段；$r_{i,t}^{L}$：负载的实际值与预测值之差，即负载预测残差；$r_{i,t}^{V}$：光伏实际发电量与预测值之差，即光伏预测残差。

3.联合场景生成

对于目标日d，将距其最近的至多S组联合残差逐一加到预测值上，从而构建等权重的场景集合：

$$L_{d,t}^{s} = \max\left(0, \hat{L}_{d,t} + r_{s,t}^{L}\right), V_{d,t}^{s} = \max\left(0, \hat{V}_{d,t} + r_{s,t}^{V}\right) \quad (14)$$

其中，$s = 1,2,\ldots,S$，$S \leq 7$。每组场景同时包含负载和光伏两条曲线，从而保留负载与光伏的日内相关性及跨时段关系。$\max(0,\cdot)$用于避免生成负的负载或光伏电量。

5.2.3 日前两阶段随机优化模型

目标函数

日前优化的目标是使计划购电费用、紧急购电期望费用以及终端库存不足惩罚之和最小：

$$\min Z_d = \sum_{t=1}^{144} p_t\, G_{d,t} + \frac{1}{S}\sum_{s=1}^{S}\sum_{t=1}^{144} 5\, p_t e_{d,t}^{s} + \kappa\left(6000 - E_{d,144}^{ref}\right)_{+} \quad (12)$$

其中，$G_{d,t}$为第d天第t时段的普通计划购电量，$e_{d,t}^{s}$为场景s下第t时段的紧急购电量，$E_{d,144}^{ref}$为第d天 24:00 的参考储电量，κ为日末储能不足惩罚系数，$(x)_{+}$为正部运算，具体含义是$(x)_{+} = \max(x, 0)$。式（12）中各部分含义如下：

$\sum_t p_t\, G_{d,t}$表示全天普通计划购电费用；

$\frac{1}{S}\sum_s \sum_t 5\, p_t e_{d,t}^{s}$表示历史场景平均紧急购电费用；

$\kappa\left(6000 - E_{d,144}^{ref}\right)_{+}$保证场景的非负性，避免日前计划过度消耗日末库存，该项不计入实际购电费用。

场景功率平衡约束

对于每个场景s，每个时段的电能供需满足：

$$G_{d,t} + V_{d,t}^{s} + b_{d,t}^{ref} + e_{d,t}^{s} = L_{d,t}^{s} + c_{d,t}^{ref} + w_{d,t}^{s} \quad (13)$$

其中，$V_{d,t}^{s}$表示s场景下的光伏电量，$b_{d,t}^{ref}$表示日前参考放电量，$c_{d,t}^{ref}$表示日前参考充电量，$w_{d,t}^{s}$表示场景s下的未利用电量。式（2）左侧分别为光伏、计划购电、储能放电和紧急购电提供的电能，右侧为负载、储能充电和未利用电量。未利用电量$w_{d,t}^{s}$包含光伏剩余电量以及已经付费但未被实际使用的计划购电量，而非纯弃光电量。

储能状态递推约束

储能设备的内部电量满足：

$$E_{d,t}^{ref} = E_{d,t-1}^{ref} + 0.9c_{d,t}^{ref} - \frac{b_{d,t}^{ref}}{0.9}, t = 1,2,\dots,144. \quad \textbf{(14)}$$

其中，充电效率和放电效率分别取 0.9。该式表示充电时实际进入电池的电量为$\boldsymbol{0.9c_{d,t}^{ref}}$，放电时为了向母线侧释放$\boldsymbol{b_{d,t}^{ref}}$的电能，电池内部需要消耗$\boldsymbol{b_{d,t}^{ref}/0.9}$。

储能容量与充放电功率约束

为保障储能系统安全运行与调度方案的物理可行性，储能电量必须保持在允许范围内：

$$1200 \le E_{d,t}^{ref} \le 10800 \quad (15)$$

单个 10 分钟时段的最大充放电功率为 5000 kW，对应最大充放电量为 \(5000/6\) kWh，因此：

$$0 \le c_{d,t}^{ref} \le \frac{5000}{6}, 0 \le b_{d,t}^{ref} \le \frac{5000}{6} \quad (16)$$

为避免在某一时段内同时充电和放电，原模型使用 0-1 变量约束：

$$0 \le c_{d,t}^{ref} \le \frac{5000}{6} z_{d,t}, \quad (17)$$

$$0 \le b_{d,t}^{ref} \le \frac{5000}{6}\left(1 - z_{d,t}\right), z_{d,t} \in \{0,1\} \quad (18)$$

同时，储能充电只能使用普通购电和光伏电量，避免通过紧急购电获取套利收益，因此有：

$$c_{d,t}^{ref} \le G_{d,t} + \min_{s} V_{d,t}^{s} \quad (19)$$

所有购电、充电、放电和未利用电量均满足非负约束：

$$G_{d,t}, c_{d,t}^{ref}, b_{d,t}^{ref}, e_{d,t}^{s}, w_{d,t}^{s} \ge 0 \quad (20)$$

问题二不要求每一天的初始储电量与日末储电量相等，但要求储能状态跨日连续：

$$E_{d,0}^{ref} = E_{d-1,144}^{act} \quad (21)$$

其中$E_{d,0}^{ref}$：第d天 0:00 的日前参考储电量；$E_{d-1,144}^{act}$：第$d-1$天 24:00 实际执行结束后的储电量。式（21）说明当天制定日前计划时的初始储电量，应取前一天实际运行结束时的储电量。

5.2.4 模型的求解方法

1.日前模型求解

充电模式为 0–1 变量，直接保留式（12）和式（13）会形成混合整数模型。由于本文允许未利用电能免费计入$w_{d,t}^{s}$，可以证明存在满足充放电互斥的等价最优解。

若某时段同时出现

$$c_{d,t}^{ref} > 0, b_{d,t}^{ref} > 0,$$

取

$$\delta = \min\left(c_{d,t}^{ref}, \frac{b_{d,t}^{ref}}{0.9^2}\right),$$

并令

$$c'_{d,t} = c_{d,t}^{ref} - \delta,$$

$$b'_{d,t} = b_{d,t}^{ref} - 0.9^2\delta,$$

此步骤将同一时段同时出现的充、放电量按效率比例同步削减，直至至少一方为零。充电减少δ使内部电量少增0.9δ，放电减少0.81δ使内部电量少耗$\frac{0.81\delta}{0.9} = 0.9\delta$，二者恰好抵消，因此储能状态保持不变，同时消除了无效充放循环。

变换后的储能状态保持不变：

$$0.9c'_{d,t} - \frac{b'_{d,t}}{0.9} = 0.9c_{d,t}^{ref} - \frac{b_{d,t}^{ref}}{0.9}.$$

同时，未利用电量增加$(1 - 0.9^2)\delta$，不会增加计划购电量和紧急购电费用。因此，可以先求解取消充放互斥变量后的线性规划模型，再消除无效充放循环，得到满足约束的调度方案。

求解结束后，每天 0:00 只固定普通计划购电量$G_{d,t}$，储能参考轨迹允许在日内根据实际观测进行修正。

2.日内储能反馈

设当前时段初储电量为E，下一时段初储电量为E'，令

$$u = E' - E.$$

对应母线侧充电量和放电量为

$$c(u) = \frac{u_+}{0.9}, b(u) = 0.9(-u)_+,$$

其中

$$u_+ = \max(u, 0).$$

当前时段的供电缺口为

$$D_t(u) = \max[L_t - V_t - G_t + c(u) - b(u), 0]. \quad (21)$$

若$D_t(u) > 0$，则需要按照$5p_t$的价格紧急购电。

本文以 100 kWh 为间隔，将储能状态从 1200 kWh 到 10800 kWh 离散化，并从日末向前构造价值函数：

$$F_t(E) = \min_{E' \in \mathcal{A}_t(E)} \left[\frac{1}{S}\sum_{s=1}^{S} 5\,p_t\left(L_t^s - V_t^s - G_t + c(E' - E) - b(E' - E)\right)_+ + F_{t+1}(E')\right], \quad (22)$$

其中，$\mathcal{A}_t(E)$是满足容量、功率和充电来源约束的可达状态集合。

终端价值函数为

$$F_{145}(E) = \kappa \max(6000 - E, 0). \tag{23}$$

当天实际负载和光伏到达后，利用实际供需选择使“当前紧急购电费用＋下一状态价值”最小的储能动作。该过程只使用当前及历史信息，不使用当天剩余时段的真实供需。

实际购电费用按照下式结算：

$$J_d = \sum_{t=1}^{144} p_t G_{d,t} + \sum_{t=1}^{144} 5\, p_t e_{d,t}. \tag{24}$$

5.2.5 求解结果

1.全年结果

评价区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。2025 年 1 月作为模型预热阶段，其购电费用单独记录，不计入正式评价结果。总实际购电费用为 13,672,895.40 元，其中计划费用 13,227,687.23 元、紧急费用为 445,208.17 元具体结果如下

表 4 全年购电策略评价

指标	结果
计划购电量	21,755,323.566396 kWh
紧急购电量	91,070.382688 kWh
实际外网购电总量	21,846,393.949085 kWh
计划购电费用	13,227,687.23 元
紧急购电费用	445,208.17 元
总实际购电费用	13,672,895.40 元
总充电量	6,509,852.649765 kWh
总放电量	5,273,233.282089 kWh
未利用电量	2,670,585.086442 kWh
发生紧急购电的 10 分钟时段数	1029
合并后的紧急购电事件数	353
2 月 1 日 0:00 储电量	6,200.000000 kWh
12 月 31 日 24:00 储电量	5,919.293578 kWh
1 月预热费用	2,158,433.86 元

其中，紧急购电量占实际外网购电量的比例约为

$$\frac{91{,}070.382688}{21{,}846{,}393.949085} \approx 0.417\%.$$

紧急购电费用占总购电费用的比例约为

$$\frac{445{,}208.17}{13{,}672{,}895.40} \approx 3.26\%.$$

2.四个指定日结果

表 4 给出四个指定日的计划与紧急购电量及费用。12 月 21 日计划购电量最高但无应急；9 月 23 日因应急费用较高，总费用最大。

四个指定日期的购电结果如下：

表 5 指定日期购电结果

日期	计划购电量/kWh	紧急购电量/kWh	计划购电费/元	紧急购电费/元	总费用/元
2025-03-20	70,307.476749	124.064066	42,525.89	630.70	43,156.59
2025-06-21	37,586.768142	0.000000	21,511.15	0.00	21,511.15
2025-09-23	67,724.429921	787.067857	41,802.47	4,988.91	46,791.38
2025-12-21	98,061.338979	0.000000	62,962.38	0.00	62,962.38

该表 5 列出四个指定日 0:00 与 24:00 的储电量。除 6 月 21 日、12 月 21 日首末从日期看，应急集中于 3 月 20 日和 9 月 23 日，6 月、12 月为零；从时段看，多发生于 17 时以后，以 9 月 23 日晚间应急量最大。

相等外，其余日期均有小幅变化，体现储能跨日连续运行。四个指定日期的 0:00 和 24:00 储电量如下：

表 6 指定日期时和 24 时储电量

日期	0:00 储电量/kWh	24:00 储电量/kWh
2025-03-20	6,200.000000	6,147.486035
2025-06-21	6,197.709417	6,200.000000
2025-09-23	5,979.173955	6,093.418637
2025-12-21	6,200.000000	6,200.000000

图 6 中从日期看，应急集中于 3 月 20 日和 9 月 23 日，6 月、12 月为零；从时段看，多发生于 17 时以后，以 9 月 23 日晚间应急量最大。紧急购电事件如下：

表 7 紧急购电事件

日期	时段	紧急购电量/kWh
2025-03-20	16:00—16:50	77.373040
2025-03-20	17:00—17:10	10.701919
2025-03-20	20:20—20:30	35.989107
2025-06-21	无	0.000000
2025-09-23	17:00—17:10	11.807592

2025-09-23	17:30—17:40	33.393467
2025-09-23	20:10—20:40	631.353112
2025-09-23	20:50—21:10	92.779446
2025-09-23	21:20—21:30	17.734241
2025-12-21	无	0.000000

由结果可见，3 月 20 日和 9 月 23 日仍存在少量紧急购电；6 月 21 日和 12 月 21 日没有发生紧急购电。采用相同星期几预测后，周六和周日负荷差异得到保留，12 月 21 日的计划购电量能够覆盖实际供需，未再产生旧模型中的大额紧急购电。

5.3 问题三模型的建立与求解

5.3.1 问题分析与预测建模

问题三在问题二的基础上增加了 0:00、6:00、12:00 和 18:00 发布的未来 24 小时光伏预报。微网在 0:00 制定普通购电计划，并在 6:00、12:00 和 18:00 根据最新预报调整尚未执行时段的购电量。如果供电仍然不足，则按照交易时刻电价的 5 倍进行紧急购电。

将一天划分为 144 个 10 分钟时段，每个时段长度为 Δt=1/6 h。记第 d 天在时刻 τ 发布、发布时间后第 h 小时的整点光伏功率预报为 $\hat{P}(d,h;\tau)$，其中 $\tau\in\{0,6,12,18\}$，$h=0,1,\ldots,24$。τ 表示预报发布时刻，而不是第几次发布。

$$\hat{P}_d^{\tau}(u)=\hat{P}_{d,h}^{\tau}+(u-h)(\hat{P}_{d,h+1}^{\tau}-\hat{P}_{d,h}^{\tau}),\quad h\le u\le h+1 \tag{25}$$

式（25）在相邻两个整点预报之间进行线性插值，从而得到连续的预报曲线。再将插值曲线在 10 分钟区间内积分，即可得到各时段的光伏预报电量。当天调度只使用到当天 24:00 的预报部分，跨次日预报不提前纳入当天模型。

$$\hat{L}_{d,t}=\frac{1}{|H_d|}\sum_{i\in H_d}L_{i,t} \tag{26}$$

负载预测采用最近 8 个相同星期几的历史日期，周一至周日分别匹配，不将周六和周日合并为同一类。Hd 为目标日之前最近 8 个相同星期几的历史日期集合。

$$r_{i,t}^{L}=L_{i,t}-\hat{L}_{i,t},\quad r_{i,t}^{V,\tau}=V_{i,t}-\hat{V}_{i,t}^{\tau} \tag{27}$$

式（27）分别定义负载预测残差和光伏预报残差。负载残差来自只使用历史信息形成的因果预测，光伏残差来自当时已经发布的有效预报版本。

$$L_{d,t}^{s}=max(0,\hat{L}_{d,t}+r_{s,t}^{L}),\quad V_{d,t}^{s,\tau}=max(0,\hat{V}_{d,t}^{\tau}+r_{s,t}^{V,\tau}) \tag{28}$$

将同一历史日的负载和光伏残差整体叠加到目标日预测上，得到最多 7 个等权联合场景。这样可以保留负载与光伏之间的相关关系，并避免分别独立扰动两类数据。

图 19—图 21 分别展示不同发布时刻的预报与实际功率、共同目标时段的预报误差以及日前预报与实际光伏日总电量的比较结果。

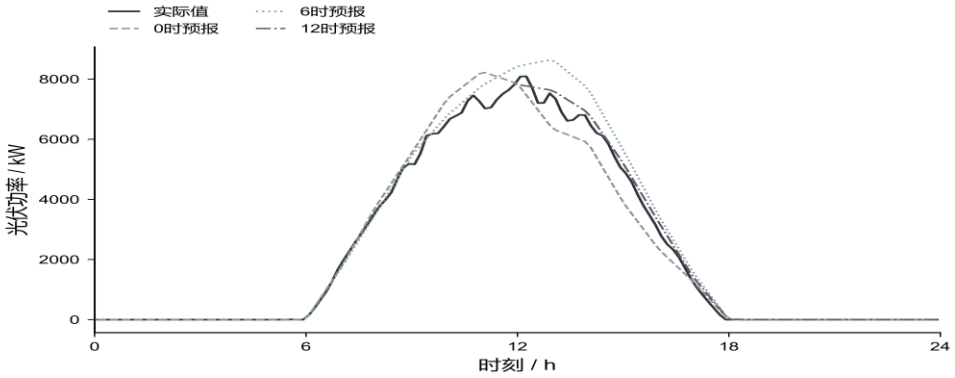


图 19 多时刻光伏预报与实际功率

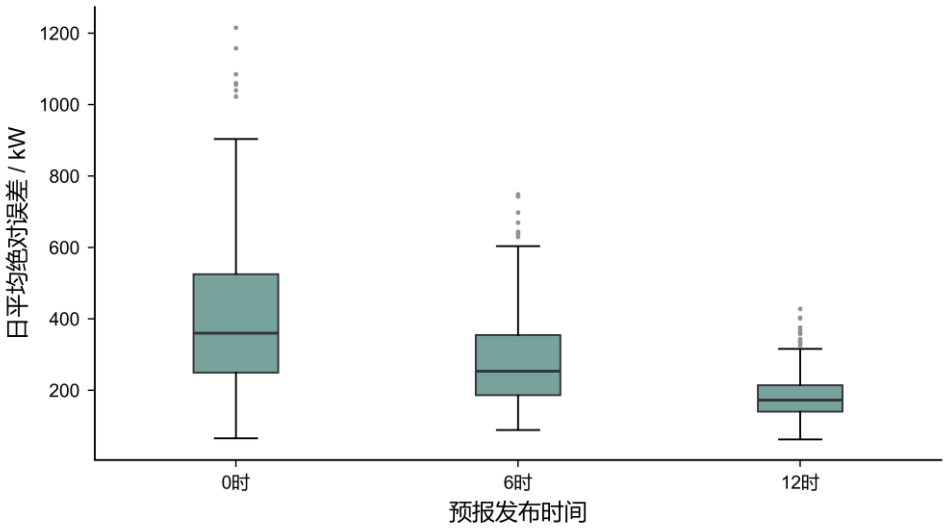


图 20 共同目标时段的预报误差分布

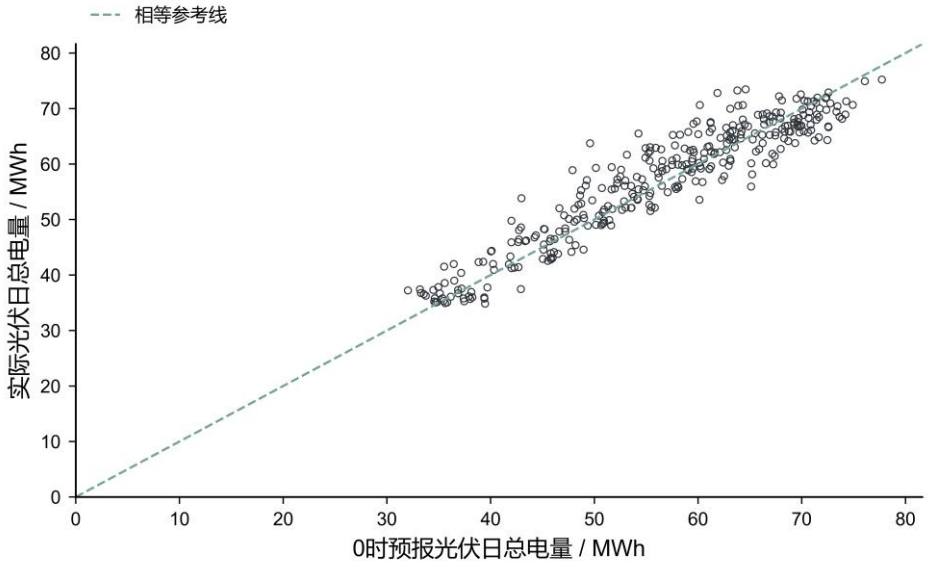


图 21 日前预报与实际光伏日总电量

5.3.2 多时点购电优化模型

设 Gd,t 为 0:00 制定的原计划购电量，Ad,t 为调整后的最终普通购电量。相对原计划的增加量和减少量分别定义为：

$$u_{d,t} = (A_{d,t} - G_{d,t})_+, \quad v_{d,t} = (G_{d,t} - A_{d,t})_+ \tag{29}$$

其中，(x)+=max(x,0)。主结算模型采用原计划购电量全额计费、最终调整净差只结算一次的口径：

$$J_d = \sum_{t=1}^{144} [p_t G_{d,t} + 1.5 p_t u_{d,t} + 0.5 p_t v_{d,t} + 5 p_t e_{d,t}] \tag{30}$$

式（30）中，第一项为原计划购电费用，第二项为增加购电费用，第三项为减少购电违约费用，第四项为紧急购电费用。减少购电时先退还原电费再收取 50%违约费的结算方式仅作为敏感性口径，不与主模型结果混用。

0:00 采用两阶段随机优化。第一阶段确定所有场景共享的计划购电量 Gd,t 及储能参考轨迹；第二阶段针对各场景确定最终调整购电量、紧急购电量和未利用电量。设场景总数为 S，则期望费用目标为：

$$min\ Z_d = \frac{1}{S} \sum_{s=1}^{S} \sum_{t=1}^{144} [p_t G_{d,t} + 1.5 p_t u_{d,t}^s + 0.5 p_t v_{d,t}^s + 5 p_t e_{d,t}^s] + \kappa (6000 - E_{d,144})_+ \tag{31}$$

终端库存惩罚系数取 κ=p1/0.9。该项仅用于日前决策，不计入实际购电费用。0:00 至 6:00 没有新的调整节点，该时段普通购电量保持不变，缺口只能通过 5 倍价格的紧急购电满足。

$$A_{d,t}^s + V_{d,t}^{s,\tau} + b_{d,t} + e_{d,t}^s = L_{d,t}^s + c_{d,t} + w_{d,t}^s \tag{32}$$

$$E_{d,t} = E_{d,t-1} + 0.9 c_{d,t} - \frac{b_{d,t}}{0.9} \tag{33}$$

式（32）为各场景的供需平衡约束，式（33）为储能状态递推约束。储能容量、功率和充放电互斥约束为：

$$1200 \le E_{d,t} \le 10800, \quad 0 \le c_{d,t}, b_{d,t} \le \frac{5000}{6}, \quad c_{d,t} b_{d,t} = 0 \tag{34}$$

此外，储能充电只能使用普通购电和光伏，不能利用紧急购电进行套利：

$$c_{d,t} \le A_{d,t}^{s} + min_s V_{d,t}^{s,\tau} \tag{35}$$

问题三采用非对称调整结算。多购买 1 kWh 需要预先支付且不退款，少购买 1 kWh 可在事后按 1.5 倍价格补充。在单个可调整时段中，若忽略储能和跨时段耦合，0:00 最优计划量近似满足：

$$P(D_t^s > G_t) = \frac{1}{1.5} = \frac{2}{3}, \quad G_t \approx D_t\text{的}33\%\text{分位数} \tag{36}$$

对于 0:00 至 6:00 没有调整机会的时段，缺口按 5 倍价格应急，对应较高分位数。最终计划量仍由包含储能和跨时段耦合的随机优化统一求解。

5.3.3 滚动调整与储能反馈求解

在 6:00、12:00 和 18:00，模型固定已经执行的历史状态，使用当前最新预报重新构造剩余时段场景，并优化尚未执行时段的最终普通购电量、储能轨迹和应急量。每次只执行当前决策节点到下一节点之间的动作，到达下一节点后重新求解。

设当前时段初储电量为 E，下一时段初储电量为 E′，令 u=E′−E，则对应的母线侧充电量和放电量分别为：

$$c(u) = \frac{u_+}{0.9}, \quad b(u) = 0.9(-u)_+ \tag{37}$$

第 t 时段的供电缺口为：

$$D_t(u) = max[L_t - V_t - A_t + c(u) - b(u), 0] \tag{38}$$

储能价值函数采用 100 kWh 状态网格，从日末向前递推：

$$F_t(E) = min_{E' \in A_t(E)} [\frac{1}{S} \sum_{s=1}^{S} 5 p_t (L_t^s - V_t^s - A_t + c(E' - E) - b(E' - E))_+ + F_{t+1}(E')] \tag{39}$$

$$F_{145}(E) = \kappa(6000 - E)_+ \tag{40}$$

日内每 10 分钟根据实际负载、光伏和储能状态选择一个充放电动作，并更新储能状态。该过程只使用当前观测、历史场景和已经发布的预报，不使用当天剩余时段的实际数据。

由于允许剩余电能免费计入未利用电量，可以先将充放电互斥约束松弛为线性规划，再通过取 δ=min(c,b/0.9²)，并令 c′=c−δ、b′=b−0.9²δ 消除同时充放电。该变换保持储能状态增量不变，可以得到满足互斥条件的可行解。图 22—图 24 分别展示原计划与最终调整购电量、日内更新与实际储电量以及不同决策时刻的最终增购量。

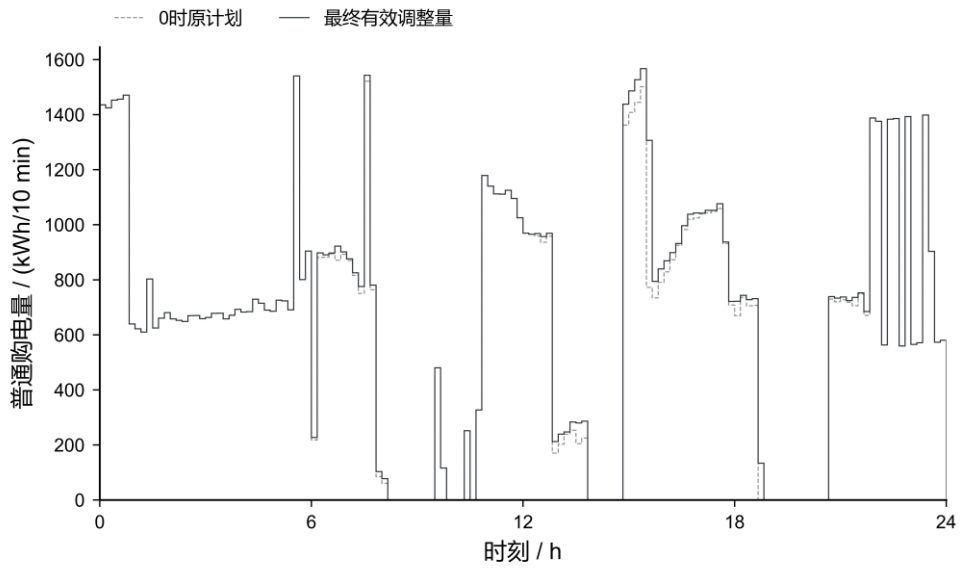


图 22 原计划与最终调整购电量

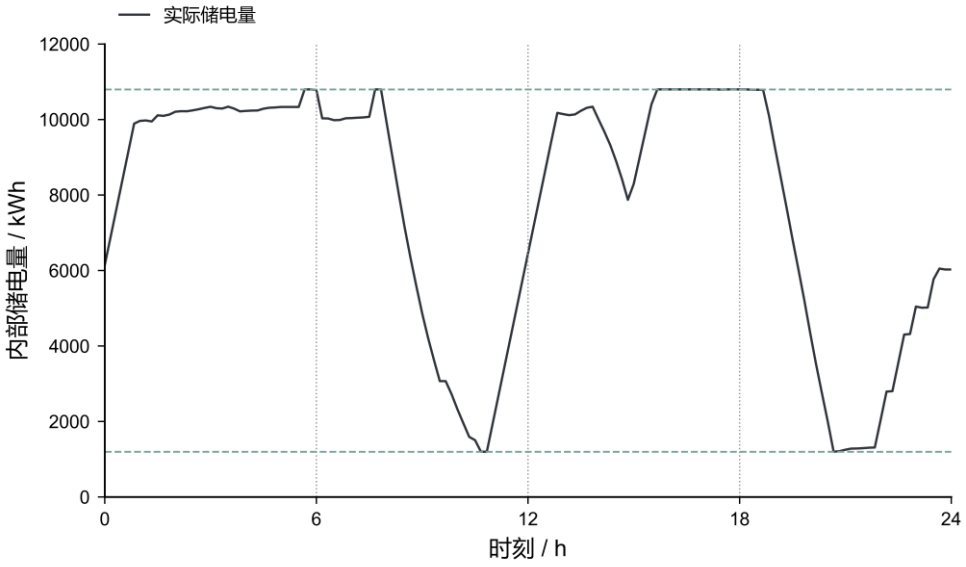


图 23 日内更新与实际储电量

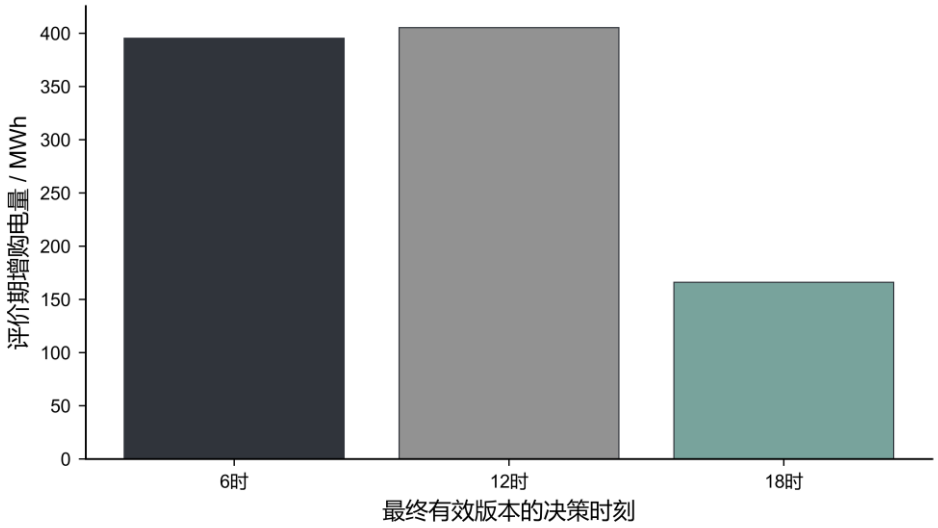


图 24 不同决策时刻的最终增购电量

5.3.4 求解结果

评价区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。2025 年 1 月作为预热阶段，不计入正式费用。全年求解结果如表 6 所示。

表 8 问题三全年购电结果

项目	结果
原计划购电费用/元	12,141,305.99
增加购电费用/元	1,116,320.27
减少购电违约费用/元	0.00
紧急购电费用/元	317,958.81
总购电费用/元	13,575,585.07
原计划购电量/kWh	20,208,033.888
最终普通购电量/kWh	21,174,733.670
紧急购电量/kWh	63,751.538
评价期初储电量/kWh	6,288.059
评价期末储电量/kWh	5,705.840

主结算口径下，原计划购电量已经全额付费，主动减少原计划还需支付违约费用，因此主策略不主动降低原计划，减少购电违约费用为 0。

表 9 四个指定日期的费用结果

日期	原计划费用/元	调整相关费用/元	紧急费用/元	总费用/元
2025-03-20	40,216.19	1,809.46	379.28	42,404.93
2025-06-21	19,157.91	1,684.51	0.00	20,842.42
2025-09-23	40,075.32	2,431.84	3,276.06	45,783.23
2025-12-21	59,486.15	2,507.77	905.09	62,899.01

四个指定日的全天普通购电量、紧急购电量和实际外网供电量如表 8 所示。

表 10 四个指定日期的购电电量

日期	原计划 G/kWh	最终普通购电 A/kWh	紧急购电 e/kWh	实际外网供电 A+e/kWh
2025-03-20	66,457.996	67,799.105	76.578	67,875.683
2025-06-21	34,449.357	35,456.306	0.000	35,456.306
2025-09-23	65,634.896	67,430.541	584.312	68,014.853
2025-12-21	93,984.006	95,937.665	261.500	96,199.165

按题目表 1 格式，指定时段的 0:00 计划购电量如表 9 所示。

表 10_1 指定时段计划购电量

时间段	2025-03-20	2025-06-21	2025-09-23	2025-12-21
10:00—10:10	0.000	0.000	0.000	0.000
12:00—12:10	502.853	0.000	267.553	969.543
14:00—14:10	0.000	0.000	0.000	0.000
16:00—16:10	654.225	128.338	461.460	829.067
18:00—18:10	563.667	0.000	747.816	669.568
20:00—20:10	0.000	0.000	0.000	0.000

按题目表 2 格式，四个指定日的四小时充放电量和日初、日末储电量如表 10 和表 11 所示。

表 11 指定日期四小时充放电量

边界时刻	2025-03-20	2025-06-21	2025-09-23	2025-12-21
00:00	**6,185.754**	**6,114.898**	**5,683.795**	**6,143.407**
24:00	**5,958.367**	**6,200.000**	**5,831.275**	**6,028.084**

按题目表 3 格式，指定日期的紧急购电结果如表 12 所示。连续正应急时段合并，不同日期之间不合并，工程零阈值为 10^{-6} kWh。图 25 给出月度购电费用构成。

表 12 指定日期紧急购电结果

日期	时间段	紧急购电量/kWh
2025-03-20	03:20—03:40	19.275
2025-03-20	09:50—10:00	28.005
2025-03-20	20:20—20:30	29.298
2025-06-21	无	0.000
2025-09-23	14:50—15:00	24.228
2025-09-23	15:50—16:10	91.085
2025-09-23	20:10—20:40	301.961
2025-09-23	20:50—21:50	167.038
2025-12-21	07:50—08:00	3.416
2025-12-21	10:30—11:30	257.219
2025-12-21	20:20—20:40	0.865

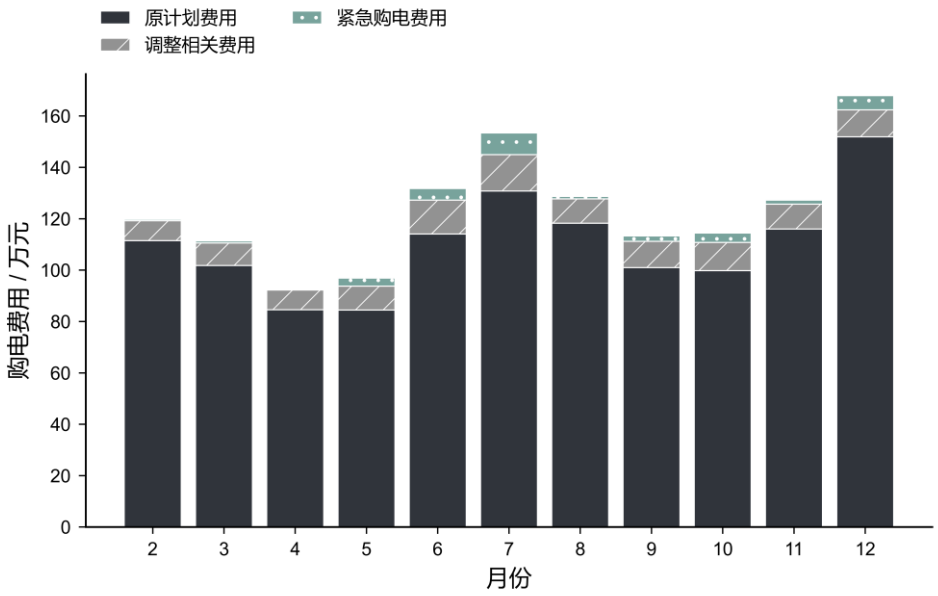


图 25 月度购电费用构成

5.3.5 预报更新价值与模型检验

为回答问题三“是否需要引入其他时刻预报”，本文设置六种策略进行对照，包括仅 0 时、0/6 时、0/6/12 时、0/6/12/18 时全部更新、四次决策均使用 0 时预报以及 18 时沿用 12 时预报。所有策略均从 2025 年 2 月 1 日 6000 kWh 开始，并要求 12 月 31 日恢复到 6000 kWh。公平边界结果如表 13 所示。

表 13 不同预报和调整策略的公平边界结果

策略	总费用/元	紧急电量/kWh
仅 0 时	13,899,899.21	93,100.474
0/6 时	13,884,179.29	90,899.155
0/6/12 时	13,625,773.35	76,492.888
0/6/12/18 时全部更新	13,576,084.03	63,889.273
四次决策均用 0 时预报	14,096,010.76	140,221.609
18 时沿用 12 时预报	13,577,240.42	64,088.202

从仅 0 时到全部更新节省 323815.18 元，但该比较同时包含新增调整机会和新预报信息。在四次调整机会相同的条件下，全部使用最新预报比始终使用 0 时预报节省 519926.73 元，说明日内新预报具有实际经济价值。12 时更新带来的费用下降较明显；18 时使用新预报相对沿用 12 时预报仅节省 1156.39 元，其重采样区间跨零，说明 18 时预报的边际收益不稳定。因此，本数据支持保留 6:00 和 12:00 预报更新，18:00 更新可根据系统接入成本决定是否保留。图 26 和图 27 分别展示相同起止库存下各策略的费用以及预报更新策略的配对费用差。

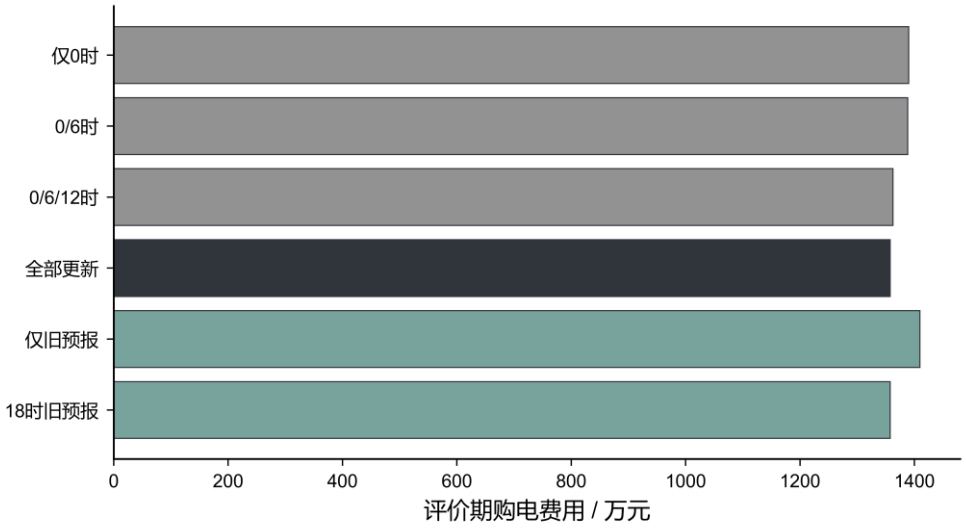


图 26 相同起止库存下的策略费用

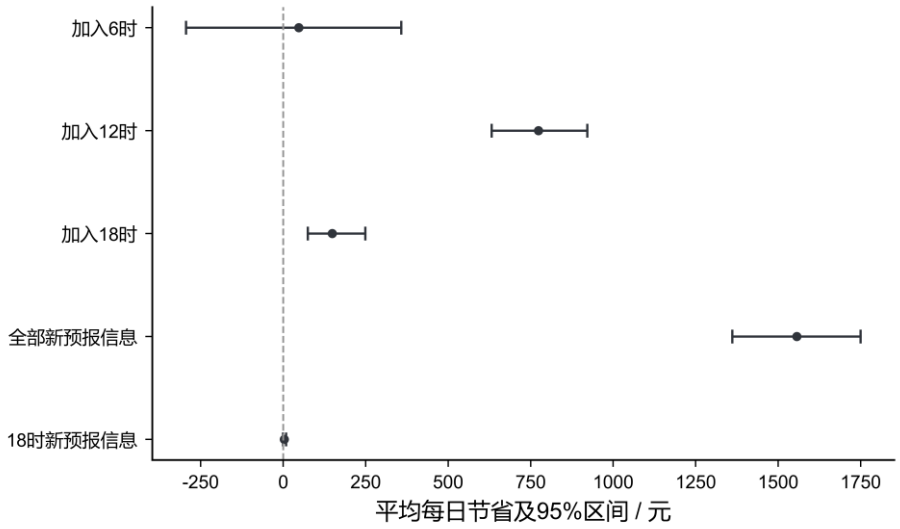


图 27 预报更新与调整策略的配对费用差

在主策略相同期初、期末库存下，完美预知联合线性规划费用为 12230108.36 元，主策略为 13575585.07 元，差额 1345476.71 元，约占 9.91%。该值是任何因果策略都无法达到的信息完全已知下界，不是实际可执行策略结果。

模型检验结果表明，最大供需平衡残差为 2.27×10^{-13} kWh，储能状态递推和跨日连续性残差为 0，不存在同时充放电，最大功率超差仅为 5.46×10^{-12} kW。该模型满足供需平衡、储能容量、功率约束和因果信息约束，但属于滚动近似策略，不构成一般多阶段随机规划问题的全局最优解。

5.4 问题四模型的建立与求解

5.4.1 问题分析与预测建模

问题四要求在实际波动电价下重新求解问题二和问题三。本文将两个分支分别记为 Q4-2 和 Q4-3：Q4-2 在每天 0:00 确定普通购电计划，日内不再调整普通购电量；Q4-3 允许在 6:00、12:00 和 18:00 根据最新光伏预报及已观测电价调整尚未执行时段。未来实际负载、光伏和电价均不能在决策前获得，实际购电按照附件 4 中供电时段的实际电价结算。

将一天划分为 144 个 10 分钟时段，时段长度为 Δt=1/6 h。负载功率和光伏功率转换为时段电量如下：

$$L_{d,t} = \frac{P^{L}_{d,t}}{6}, \quad V_{d,t} = \frac{P^{V}_{d,t}}{6} \tag{41}$$

负载以及 Q4-2 分支的光伏预测采用最近 8 个相同星期几的历史均值，周一至周日分别匹配。

$$\hat{L}_{d,t} = \frac{1}{|H_d|}\sum_{i\in H_d} L_{i,t}, \quad \hat{V}_{d,t} = \frac{1}{|H_d|}\sum_{i\in H_d} V_{i,t} \tag{42}$$

基础电价预测取过去最多 28 个完整日期在同一 10 分钟时段的平均值。在 6:00、12:00 和 18:00 更新时，利用此前最多 6 个已观测时段的价格误差进行指数衰减修正。

$$\hat{p}^{r}_{d,t} = max[0.001, \hat{p}^{0}_{d,t} + a_{d,r} exp(-\frac{t-r}{36})] \tag{43}$$

其中，r 和 t 均为 10 分钟时段编号，a 为决策节点前已经观测到的价格预测偏差均值，衰减时间尺度为 6 小时。未来实际价格不能用于当前动作。

选取此前最多 7 个完整历史日，将负载、光伏和价格的残差按照同一历史日整体叠加，构成联合场景：

$$L^{s}_{d,t} = max(0, \hat{L}_{d,t} + r^{L}_{s,t}), \quad V^{s,r}_{d,t} = max(0, \hat{V}^{r}_{d,t} + r^{V}_{s,t}), \quad p^{s,r}_{d,t} = max(0.001, \hat{p}^{r}_{d,t} + r^{p}_{s,t}) \tag{44}$$

5.4.2 波动电价下的购电优化模型

设 Gd,t 为 0 时制定的原计划购电量，Ad,t 为最终执行的普通购电量，ed,t 为紧急购电量，cd,t 和 bd,t 分别为母线侧充电量和放电量，Ed,t 为电池内部储电量，wd,t 为未利用电量。所有电量单位为 kWh，价格为元/kWh。

电能供需平衡约束为：

$$A_{d,t} + V_{d,t} + b_{d,t} + e_{d,t} = L_{d,t} + c_{d,t} + w_{d,t} \tag{45}$$

储能状态递推、容量和功率约束为：

$$E_{d,t} = E_{d,t-1} + 0.9c_{d,t} - \frac{b_{d,t}}{0.9}, \quad 1200 \le E_{d,t} \le 10800, \quad 0 \le c_{d,t}, \quad b_{d,t} \le \frac{5000}{6}, \quad c_{d,t} \tag{46}$$

储能充电只能使用普通购电和光伏，不能利用紧急购电进行套利：

$$c_{d,t} \le A_{d,t} + V_{d,t} \tag{47}$$

Q4-2 分支不允许调整普通购电量，因此 Ad,t=Gd,t，实际购电费用为：

$$J_{4-2} = \sum_{t=1}^{144} p_{d,t}(G_{d,t}+5e_{d,t}) \tag{48}$$

Q4-3 分支定义相对原计划的增加量和减少量为：

$$u_{d,t} = (A_{d,t}-G_{d,t})_+, \quad v_{d,t} = (G_{d,t}-A_{d,t})_+ \tag{49}$$

主结算口径下，原计划购电量全额计费，调整净差只结算一次，实际费用为：

$$J_{4-3} = \sum_{t=1}^{144} p_{d,t}[G_{d,t}+1.5u_{d,t}+0.5v_{d,t}+5e_{d,t}] \tag{50}$$

由于原计划不退款，在正电价条件下主策略不会主动减少原计划，因此主结果中 v=0。终端库存控制项为：

$$\Phi(E_{d,144}) = \kappa(6000 - E_{d,144})_+, \quad \kappa = \frac{\hat{p}_{d,1}}{0.9} \tag{51}$$

该终端项只用于控制跨日库存，不计入实际购电费用。0 时规划使用价格场景期望，不能用期望价格与期望应急量的乘积替代应急费用的期望。

5.4.3 滚动调整与储能反馈求解

Q4-3 分支在每个调整节点固定已经执行的历史购电和储能状态，只使用当时可获得的光伏预报和电价信息优化剩余时段。每次只执行当前节点与下一节点之间的动作，到达下一节点后重新求解。

设当前时段初储电量为 E，下一时段初储电量为 E′，令 u=E′−E，则对应的母线侧充电量和放电量为：

$$c(u) = \frac{u_+}{0.9}, \quad b(u) = 0.9(-u)_+ \tag{52}$$

当前时段的供电缺口为：

$$D_t(u) = max[L_t-V_t-A_t + c(u) - b(u),0] \tag{53}$$

储能价值函数采用 100 kWh 状态网格，从日末向前递推：

$$F_t(E) = min_{E'\in A_t(E)}[\frac{1}{S}\sum_{s=1}^{S} 5p_t^s(L_t^s-V_t^s-A_t + c(E'-E) - b(E'-E))_+ + F_{t+1}(E')] \tag{54}$$

$$F_{145}(E) = \kappa(6000 - E)_+ \tag{55}$$

日内每 10 分钟根据实际负载、光伏、当前电价和储能状态选择一个动作，不使用后续实际价格或供需数据。

若 LP 解中出现同时充放电，取 δ=min(c,b/0.9²)，并令 c′=c−δ、b′=b−0.9²δ。该变换保持储能状态增量不变，可以消除无效充放循环并得到满足物理约束的可行解。

5.4.4 求解结果

评价区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。两个分支均从 2025 年 1 月 1 日 6000 kWh 开始连续运行，1 月作为预热阶段，不计入正式费用。全年结果如表 14 所示。

表 14 问题四两分支全年购电结果

分支	原计划费用/元	增购费用/元	应急费用/元	实际总费用/元	应急电量/kWh

Q4-2	13,963,572.43	0.00	447,143.66	14,410,716.10	79,181.70
Q4-3	14,145,174.94	224,037.716	165,757.05	14,534,969.71	27,130.03

Q4-3 相对 Q4-2 的总费用增加 124,253.61 元，但应急购电量减少 52,051.68 kWh。说明增加日内调整能够降低应急电量，但在当前共享轨迹和保守日前计划下，并不必然降低总购电费用。

Q4-2 的四个指定日结果如表 15 所示。

表 15 Q4-2 四个指定日购电结果

日期	原计划 G/kWh	最终普通 A/kWh	应急 e/kWh	总费用/元
2025-03-20	71,049.648	71,049.648	109.172	44,978.255
2025-06-21	35,912.806	35,912.806	0.000000	18,124.323
2025-09-23	67,689.714	67,689.714	791.641	48,658.328
2025-12-21	97,624.687	97,624.687	0.000000	73,795.257

Q4-3 的四个指定日结果如表 16 所示。

表 16 Q4-3 四个指定日购电结果

日期	原计划 G/kWh	最终普通 A/kWh	应急 e/kWh	总费用/元
03-20	72,853.534748	73,164.714848	10.504077	46,459.17
06-21	34,745.556112	34,804.063429	0.000000	17,716.29
09-23	72,449.177191	72,465.298766	187.702485	47,999.78
12-21	98,721.689655	98,761.542870	3.415640	74,799.80

与 Q4-2 相比，Q4-3 通过调整普通购电量明显减少了应急购电。两个分支的完整 144 时段计划、充放电量和应急事件分别保存在 result4-2.xlsx 和 result4-3.xlsx 中。

5.4.5 模型检验与信息价值分析

两分支的供需平衡最大残差为 2.27×10^{-13} kWh，储能状态递推、跨日连续性、充放电互斥和充电来源残差均为 0，功率上限最大数值超差为 5.46×10^{-12} kW，所有结果满足物理约束和因果信息约束。

为区分增加调整机会与增加光伏预报信息的作用，在统一期初、期末库存为 6000 kWh 的条件下对 Q4-3 设置对照策略，结果如表 17 所示。

表 17 Q4-3 不同预报与调整策略的公平边界结果

策略	公平边界总费用/元	应急电量/kWh
Q4-2	14,410,928.89	79,188.629304
Q4-3 全部更新	14,535,182.50	27,136.951219
Q4-3 仅 0 时调整	14,662,380.93	89,434.449635
Q4-3 仅使用 0 时预报	14,536,420.25	53,133.424409
Q4-3 的 18 时沿用 12 时预报	14,535,447.04	27,202.061743

仅 0 时调整到全部更新可节省 127,198.43 元，但该比较同时包含增加调整机会和增加预报信息。固定调整机会后，使用最新光伏预报相对始终使用 0 时预报仅节省 1,237.75 元，日均 3.71 元，其 95%描述性区间包含负值，因此不能证明日内新预报在当前策略下具有稳定经济收益。18 时新预报相对沿用 12 时预报的日均节省 0.79 元，区间同样跨零。

完美预知全部实际价格、负载和光伏时，联合线性规划下界为 12,782,834.194582 元。Q4-2 实际费用与该下界相差 1,627,881.91 元，Q4-3 相差 1,752,135.52 元。该下界不可实际实现，只能用于衡量信息不完全和近似策略造成的优化空间。

将储能网格由 100 kWh 加密到 50 kWh 后，四个指定日结果变化较小。将充放电效率改为两侧均取√0.9 后费用下降，属于 90%效率定义的另一种物理解释，不能与主结果混用。

综上，问题四建立了波动电价下的日前计划、滚动调整和储能反馈模型。两分支均满足系统物理约束；Q4-3 能够显著降低应急电量，但当前近似策略没有表现出稳定的费用优势，后续可通过更完整的多阶段随机优化进一步利用日内信息。

6、模型检验

（1）误差分析：

本文模型的随机性只来自输入预测，故精度检验聚焦预测层，并以求解层的数值残差佐证算法实现无误。在评价期 48,096 个时段上，负载预测的 RMSE 为 56.653 kWh/段、MAE 为 41.525 kWh/段、MAPE 为 5.81%（WAPE 5.40%），两个口径接近，说明误差在昼夜分布均匀、无系统性偏移。

（2）灵敏度分析：

综合来看，除终端目标库存这一本身即属决策口径的参数外，核心参数在一个数量级内的变动所导致的费用偏差均控制在 5% 左右，方案结构（何时充、何时放）不发生方向性改变。

7、模型优缺点评价

7.1 模型的优点

本模型基于合理的设和现实中的约束条件，综合了供需不确定性、储能动态约束等因素，模型能够适应不同预报更新频率、交易规则和储能运行条件，具有较好的普适性。

模型利用联合场景、滚动调整和储能反馈应对预测偏差，并通过参数敏感性与公平边界检验，主要结论稳定，鲁棒性较强。

7.2 模型的缺点

由于系统充分考虑了多时点光伏预报更新、非对称调整结算和储能跨时段耦合等因素，导致模型较为复杂，计算时间长，效率低。

未考虑极端天气、自然灾害等不可控因素，这可能对实际调控策略影响较大。

7.3 模型的改进

本文通过多时点预报更新、滚动调整和非对称结算提升了模型适应性，但仍可在完整多阶段随机规划、电池寿命成本和极端风险控制等方面进一步改进。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试等，详细使用情况见支撑材料。

参考文献

[1] 刘仕豪,吴耕纬,赵书强,等.计及微网对外供电的配电网可靠性评估[J].华北电力大学学报(自然科学版),2019,46(1):9-16.

[2] 设当前时段初储电量为原亚宁,李琳,赵浩然.基于主从博弈模型的交直流混合微电网源网协调优化运行方法[J].智慧电力,2019,47(3):30-37.

附录

附录 1：solve_q1.py：

介绍：问题一，构建并求解含储能充放电约束的确定性混合整数优化模型。

```
    def optimize(p,l,v,eta=.9):
        n=len(p);nv=6*n
        # 顺序：普通购电 G、母线充电 c、母线放电 b、内部储能 E、剩余 w、充电模式 z
        g,c,b,e,w,z=[np.arange(k*n,(k+1)*n) for k in range(6)]
        obj=np.zeros(nv);obj[g]=p
        low=np.zeros(nv);high=np.full(nv,np.inf)
        low[e]=1200;high[e]=10800;high[c]=high[b]=5000/6;high[z]=1
        low[e[-1]]=high[e[-1]]=6000
        integrality=np.zeros(nv);integrality[z]=1
        mat=lil_matrix((5*n,nv));lb=np.full(5*n,-np.inf);ub=np.full(5*n,np.inf)
        for t in range(n):
            mat[t,g[t]]=1;mat[t,b[t]]=1;mat[t,c[t]]=-1;mat[t,w[t]]=-1
            lb[t]=ub[t]=l[t]-v[t]
            mat[n+t,e[t]]=1;mat[n+t,c[t]]=-eta;mat[n+t,b[t]]=1/eta
            if t:mat[n+t,e[t-1]]=-1;lb[n+t]=ub[n+t]=0
            else:lb[n+t]=ub[n+t]=6000
            mat[2*n+t,c[t]]=1;mat[2*n+t,z[t]]=-5000/6;ub[2*n+t]=0
            mat[3*n+t,b[t]]=1;mat[3*n+t,z[t]]=5000/6;ub[3*n+t]=5000/6
            mat[4*n+t,c[t]]=1;mat[4*n+t,g[t]]=-1;ub[4*n+t]=v[t]

mat=mat.tocsr();constraint=LinearConstraint(mat,lb,ub);bounds=Bounds(low,high)
        options={'mip_rel_gap':1e-10,'time_limit':120}

primary=milp(obj,integrality=integrality,bounds=bounds,constraints=constraint,options=options)
        if not primary.success:raise RuntimeError(primary.message)

relaxed=milp(obj,integrality=np.zeros(nv),bounds=bounds,constraints=constraint)
        if not relaxed.success:raise RuntimeError(relaxed.message)
        # 多解时以母线吞吐量作为第二目标
        secondary=np.zeros(nv);secondary[c]=1;secondary[b]=1
        mat2=vstack([mat,csr_matrix(obj.reshape(1,-1))],format='csr')
        res=milp(secondary,integrality=integrality,bounds=bounds,
                 constraints=LinearConstraint(mat2,np.r_[lb,-
np.inf],np.r_[ub,primary.fun+1e-7]),options=options)
        if not res.success:raise RuntimeError(res.message)
        x=res.x;mx=mat@x
```

```
        violation=max(float(np.maximum(lb-mx,0).max()),float(np.maximum(mx-
ub,0).max()),
                      float(np.maximum(low-x,0).max()),float(np.maximum(x-
high,0).max()))
        assert violation<1e-5,violation
        assert np.max(np.minimum(x[c],x[b]))<1e-5
        assert abs(x[e][-1]-6000)<1e-5
```

附录 2： solve_q2.py

介绍：问题二，实现历史预测、联合场景生成、日前购电优化及储能实时反馈控制。

```
    def predict(history, date, weeks=LOOKBACK_WEEKS):
        """只接收截至昨天的历史数组
        买应急电，周六被高估则弃电。取最近 weeks 个同一星期几的样本平均；
        历史不足 weeks 周时用全部可得样本。
        """
        n = len(history)
        assert n > 0
        ids = np.arange(n - min(n, weeks * 7), n)
        same = ids[DATES[ids].dayofweek == date.dayofweek]
        if len(same) == 0: same = ids
        return history[same].mean(axis=0)

    def scenarios(pred, residuals, history, count=7):
        if len(residuals) >= 2:
            return np.maximum(0, pred[None,:,:]+np.asarray(residuals[-count:]))
        return history[-count:].copy()

    def plan_day(p, scene, initial, eta=.9, kappa=None, hard_terminal=False):
        """共享电池参考轨迹的场景 LP。内部以 MWh 缩放，目标单位千元。"""
        if kappa is None: kappa=p[0]/eta
        s,n = scene.shape[0],len(p)
        g,c,b,e = [np.arange(k*n,(k+1)*n) for k in range(4)]
        q = np.arange(4*n,(4+s)*n).reshape(s,n)
        h=(4+s)*n; nv=h+1
        obj=np.zeros(nv);obj[g]=p;obj[q]=5*p[None,:]/s;obj[h]=kappa
        obj[c]=obj[b]=1e-8   # 小于 1 分钱的吞吐量择优扰动，另报告原始经济目
标。
        bounds=[(0,None)]*nv
        for t in range(n):
            bounds[c[t]]=bounds[b[t]]=(0,POWER/1000)
```

```
                bounds[e[t]]=(EMIN/1000,EMAX/1000)
        if hard_terminal:
                bounds[e[-1]]=(EREF/1000,EREF/1000)
                # 最后 7 段保留足够普通电源，确保实际闭环可达。
                for t in range(n-7,n): bounds[g[t]]=(POWER/1000,None)
        eq=lil_matrix((n,nv));rhs=np.zeros(n)
        for t in range(n):
                eq[t,e[t]]=1;eq[t,c[t]]=-eta;eq[t,b[t]]=1/eta
                if t: eq[t,e[t-1]]=-1
                else: rhs[t]=initial/1000
        a=lil_matrix((s*n+n+1,nv));ub=np.zeros(s*n+n+1)
        for j in range(s):
                for t in range(n):
                        row=j*n+t
                        a[row,g[t]]=-1;a[row,b[t]]=-1;a[row,c[t]]=1;a[row,q[j,t]]=-1
                        ub[row]=(scene[j,1,t]-scene[j,0,t])/1000
        for t in range(n):
                a[s*n+t,c[t]]=1;a[s*n+t,g[t]]=-1
                ub[s*n+t]=scene[:,1,t].min()/1000
        a[-1,e[-1]]=-1;a[-1,h]=-1;ub[-1]=-EREF/1000

res=linprog(obj,A_ub=a.tocsr(),b_ub=ub,A_eq=eq.tocsr(),b_eq=rhs,bounds=bounds,
                        method='highs',options={'primal_feasibility_tolerance':1e-
8,'dual_feasibility_tolerance':1e-8})
        if not res.success: raise RuntimeError(res.message)
        x=res.x*1000
        # 任意同时充放解可消去无效循环：状态不变，剩余电量增加，费用不
升。
        cancel=np.minimum(x[c],x[b]/eta**2)
        charge=np.maximum(0,x[c]-cancel); discharge=np.maximum(0,x[b]-
eta**2*cancel)
        grid=np.maximum(0,x[g]); storage=x[e]
        short=np.maximum(scene[:,0,:]+charge-grid-scene[:,1,:]-discharge,0)
        economic=float(p@grid+np.mean(short@(5*p))+kappa*max(EREF-storage[-
1],0))
        err=max(float(np.abs(np.r_[initial,storage[:-1]]+eta*charge-discharge/eta-
storage).max()),
                        float(np.maximum(charge-grid-scene[:,1,:].min(axis=0),0).max()))
        assert err < 1e-4 and np.max(np.minimum(charge,discharge))<1e-5
        return grid, storage,
{'plan_objective_yuan':economic,'lp_objective_yuan':float(res.fun*1000),
                                        'lp_iterations':int(res.nit),'plan_violation_kwh':err}

    def value_table(p, grid, scene, eta=.9, kappa=None, step=100., hard_terminal=False):
```

```
        """不冒充保留全部时序相关性的精确多阶段 DP。"""
        if kappa is None:kappa=p[0]/eta
        states=np.arange(EMIN,EMAX+step/2,step)
        assert abs(states[-1]-EMAX)<1e-8
        delta=states[None,:]-states[:,None]
        charge=np.maximum(delta,0)/eta; discharge=np.maximum(-delta,0)*eta
        bus=charge-discharge
        valid=(charge<=POWER+1e-8)&(discharge<=POWER+1e-8)
        value=np.empty((len(p)+1,len(states)))
        value[-1]=kappa*np.maximum(EREF-states,0)
        if hard_terminal:value[-1]=np.where(np.abs(states-EREF)<1e-8,0,np.inf)
        for t in range(len(p)-1,-1,-1):
            net=scene[:,0,t]-scene[:,1,t]-grid[t]

cost=np.maximum(net[:,None,None]+bus[None,:,:],0).mean(axis=0)*(5*p[t])
            # 仅约束普通电源可供充电；应急用于负载，约束与日前模型一致。
            allowed=valid & (charge<=grid[t]+scene[:,1,t].min()+1e-8)
            total=cost+value[t+1][None,:]
            total[~allowed]=np.inf
            value[t]=np.min(total,axis=1)
        return states,value

    def action(p, grid, load, pv, energy, states, future_value, eta=.9):
        """只使用本段观测，不接收当天未来实际值；储能在网格间作分段线性插
值。"""
        lo=max(EMIN,energy-POWER/eta)
        hi=min(EMAX,energy+eta*min(POWER,grid+pv))
        net=load-pv-grid
        balanced=energy-net/eta if net>=0 else energy-net*eta
        candidates=np.unique(np.r_[states[(states>=lo)&(states<=hi)],lo,hi,
                                   np.clip(energy,lo,hi),np.clip(balanced,lo,hi)])
        change=candidates-energy
        c=np.maximum(change,0)/eta;b=np.maximum(-change,0)*eta
        emergency=np.maximum(net+c-b,0)
        values=5*p*emergency+np.interp(candidates,states,future_value)
        finite=np.isfinite(values)
        if not finite.any():raise RuntimeError('No feasible real-time action')
        best=values[finite].min()
        # 数值等价时最小吞吐量；不把实际吞吐惩罚计入费用。
        ids=np.flatnonzero(finite & (values<=best+1e-7))
        j=ids[np.argmin((c+b)[ids])]
        unused=max(grid+pv+b[j]+emergency[j]-load-c[j],0)
        return float(candidates[j]),float(c[j]),float(b[j]),float(emergency[j]),float(unused)
```

```
    def execute(p, grid, scene, actual, initial, eta=.9, kappa=None, step=100.,
hard_terminal=False):
        states,value=value_table(p,grid,scene,eta,kappa,step,hard_terminal)
        energy=initial;rows=[]
        for t in range(len(p)):
            start=energy

energy,c,b,q,w=action(p[t],grid[t],actual[0,t],actual[1,t],start,states,value[t+1],eta)
            rows.append((start,energy,c,b,q,w))
        return np.array(rows)

    def baseline_grid(scene):
        # min pG+5p E[(L-V-G)+] 的经验 0.8 分位数；不含电池。
        x=np.sort(scene[:,0,:]-scene[:,1,:],axis=0)
        return np.maximum(0,x[int(np.ceil(.8*len(x)))-1])

    def run_period(p, data, days=365, lookback_weeks=LOOKBACK_WEEKS,
step=100.,
                   scene_count=7, eta=.9, start_index=0, initial=EREF,
                   hard_last=False, progress=False, return_scenes=False):
        energy=initial;residuals=[];records=[];daily=[];saved={}
        kappa=p[0]/eta
        for d in range(days):
            date=DATES[d]
            if d==0:
                pred=np.zeros((2,T));scene=None
            else:
                pred=predict(data[:d],date,lookback_weeks)
                scene=scenarios(pred,residuals,data[:d],scene_count)
            if d>=start_index:
                start=energy
                if d==0:
                    grid=np.zeros(T);ref=np.full(T,energy)

out=np.column_stack([np.full(T,energy),np.full(T,energy),np.zeros((T,2)),
                                                    np.maximum(data[d,0]-
data[d,1],0),np.maximum(data[d,1]-data[d,0],0)])

info={'plan_objective_yuan':0.,'lp_objective_yuan':0.,'lp_iterations':0,'plan_violation_kwh':
0.}
                    basegrid=np.zeros(T)
                else:
                    terminal=hard_last and d==days-1
                    grid,ref,info=plan_day(p,scene,energy,eta,kappa,terminal)
```

```
                        out=execute(p,grid,scene,data[d],energy,eta,kappa,step,terminal)
                        basegrid=baseline_grid(scene)
                    energy=out[-1,1]
                    baseq=np.maximum(data[d,0]-data[d,1]-basegrid,0)

table=pd.DataFrame(out,columns=['storage_start_kwh','storage_end_kwh','charge_kwh','dis
charge_kwh','emergency_kwh','unused_kwh'])

table.insert(0,'date',date.strftime('%Y-%m-%d'));table.insert(1,'slot',np.arange(1,T+1))
                    table['interval']=[interval(t) for t in range(T)]

table['price_yuan_per_kwh']=p;table['load_kwh']=data[d,0];table['pv_kwh']=data[d,1]
                    table['forecast_load_kwh']=pred[0];table['forecast_pv_kwh']=pred[1]
                    table['grid_kwh']=grid;table['reference_storage_kwh']=ref

table['plan_cost_yuan']=p*grid;table['emergency_cost_yuan']=5*p*table.emergency_kwh

table['baseline_grid_kwh']=basegrid;table['baseline_emergency_kwh']=baseq
                    table['baseline_cost_yuan']=p*basegrid+5*p*baseq
                    table['lookback_weeks']=lookback_weeks;table['scenarios']=0 if scene
is None else len(scene)
                    table['history_latest_date']='' if d==0 else DATES[d-
1].strftime('%Y-%m-%d')
                    records.append(table)

summary={'date':date.strftime('%Y-%m-%d'),'lookback_weeks':lookback_weeks,'scenarios'
:0 if scene is None else len(scene),
                                'initial_kwh':start,'final_kwh':energy,**info}
                    for col in
['grid_kwh','emergency_kwh','charge_kwh','discharge_kwh','unused_kwh','plan_cost_yuan','
emergency_cost_yuan','baseline_cost_yuan']:
                        summary[col]=float(table[col].sum())

summary['total_cost_yuan']=summary['plan_cost_yuan']+summary['emergency_cost_yuan'
]
                    summary['prediction_mae_kwh']=float(np.mean(np.abs(pred-data[d])))
                    summary['emergency_slots']=int((table.emergency_kwh>1e-6).sum())
                    daily.append(summary)
                    if return_scenes and str(date.date()) in SELECTED:
                        saved[str(date.date())]=(pred.copy(),[r.copy() for r in
residuals],start,d)
                    if progress and (d%20==0 or d==days-1):
                        print(f'{date.date()} completed;
cost={summary["total_cost_yuan"]:.2f}; E={energy:.2f}',flush=True)
```

```
                if d>0:residuals.append(data[d]-pred)
        return pd.concat(records,ignore_index=True),pd.DataFrame(daily),saved

    if __name__=='__main__':main()
```

附录 3：solve_q3.py

介绍：问题三，基于滚动更新的光伏预报进行场景生成和日内购电调整优化。

```
    # forecasts[d, r, t]：
    #  第 d 日、r 属于{0,6,12,18}时发布的光伏预测

    ROOT=Path(__file__).resolve().parent
    POLICIES={
        'only0':((0,),{}),
        'add6':((0,1),{}),
        'add12':((0,1,2),{}),
        'all':((0,1,2,3),{}),
        'stale_all':((0,1,2,3),{1:0,2:0,3:0}),
        'stale18':((0,1,2,3),{3:2}),
    }

    def make_scene(actual,forecasts,load_pred,d,source_release,offset,count=7):
        """每个联合残差来自同一个已经结束的历史日；日内以前发布的预报仍可取后缀。"""
        assert d>0 and source_release*36<=offset
        ids=np.arange(max(1,d-count),d)
        base=np.stack([load_pred[d,offset:],forecasts[d,source_release,offset:]])
        if len(ids)==0:return base[None,:,:]
        load=base[0]+actual[ids,0,offset:]-load_pred[ids,offset:]
        pv=base[1]+actual[ids,1,offset:]-forecasts[ids,source_release,offset:]
        scene=np.maximum(np.stack([load,pv],axis=1),0)
        assert np.isfinite(scene).all()
        return scene

    def adjust_plan(p,scene,initial,original,eta=.9,kappa=.472,refund=False,hard_terminal=False):
        """以 0 时原计划为基准结算，当前窗口原计划费用为常数，不重复计入优化目标。"""
        s,n=scene.shape[0],len(p)
        a,c,b,e=[np.arange(k*n,(k+1)*n) for k in range(4)]
```

```
        q=np.arange(4*n,(4+s)*n).reshape(s,n)
        u=np.arange((4+s)*n,(5+s)*n);v=np.arange((5+s)*n,(6+s)*n)
        h=(6+s)*n;nv=h+1
        obj=np.zeros(nv);obj[u]=1.5*p;obj[v]=(-.5 if refund else .5)*p
        obj[q]=5*p[None,:]/s;obj[h]=kappa;obj[c]=obj[b]=1e-8
        bounds=[(0,None)]*nv
        for t in range(n):
            bounds[c[t]]=bounds[b[t]]=(0,POWER/1000)
            bounds[e[t]]=(EMIN/1000,EMAX/1000)
        if hard_terminal:
            bounds[e[-1]]=(EREF/1000,EREF/1000)
            for t in range(max(0,n-7),n):bounds[a[t]]=(POWER/1000,None)
        eq=lil_matrix((2*n,nv));rhs=np.zeros(2*n)
        for t in range(n):
            eq[t,e[t]]=1;eq[t,c[t]]=-eta;eq[t,b[t]]=1/eta
            if t:eq[t,e[t-1]]=-1
            else:rhs[t]=initial/1000
            eq[n+t,a[t]]=1;eq[n+t,u[t]]=-1;eq[n+t,v[t]]=1;rhs[n+t]=original[t]/1000
        mat=lil_matrix((s*n+n+1,nv));ub=np.zeros(s*n+n+1)
        for j in range(s):
            for t in range(n):
                row=j*n+t
                mat[row,a[t]]=-1;mat[row,b[t]]=-1;mat[row,c[t]]=1;mat[row,q[j,t]]=-1
                ub[row]=(scene[j,1,t]-scene[j,0,t])/1000
        for t in range(n):
            mat[s*n+t,c[t]]=1;mat[s*n+t,a[t]]=-1;ub[s*n+t]=scene[:,1,t].min()/1000
        mat[-1,e[-1]]=-1;mat[-1,h]=-1;ub[-1]=-EREF/1000

res=linprog(obj,A_ub=mat.tocsr(),b_ub=ub,A_eq=eq.tocsr(),b_eq=rhs,bounds=bounds,met
hod='highs',
                    options={'primal_feasibility_tolerance':1e-
8,'dual_feasibility_tolerance':1e-8})
        if not res.success:raise RuntimeError(res.message)
        x=res.x*1000;grid=np.maximum(x[a],0)
        cancel=np.minimum(x[c],x[b]/eta**2)
        charge=np.maximum(x[c]-cancel,0);discharge=np.maximum(x[b]-
eta**2*cancel,0)
        assert np.max(np.abs(np.r_[initial,x[e][:-1]]+eta*charge-discharge/eta-x[e]))<1e-
4
        assert np.max(np.minimum(charge,discharge))<1e-5
        assert np.max(np.maximum(charge-grid-scene[:,1,:].min(axis=0),0))<1e-4
        if not refund:assert (grid>=original-1e-5).all(), 'Non-refund reduction contradicts
free-surplus dominance'
        return grid,x[e],float(res.fun*1000)
```

```
    def simulate_day(p,actual,forecasts,load_pred,d,initial,policy='all',step=100.,eta=.9,

count=7,kappa_multiplier=1.,refund=False,hard_terminal=False,keep_versions=False,
                             legacy_plan=False):
        active,source_override=POLICIES[policy];kappa=p[0]/eta*kappa_multiplier
        energy=initial;results=[];versions=[];grid=None;reference=None
        active_grid=np.zeros(T);original=np.zeros(T);ref_all=np.full(T,initial)
        source_all=np.zeros(T,int);decision_all=np.zeros(T,int)
        objective=[]
        if d==0:
            q=np.maximum(actual[d,0]-actual[d,1],0);w=np.maximum(actual[d,1]-
actual[d,0],0)

out=np.column_stack([np.full(T,initial),np.full(T,initial),np.zeros((T,2)),q,w])
        else:
            for t in range(T):
                r=t//36
                if t%36==0 and r in active:

source=source_override.get(r,r);scene=make_scene(actual,forecasts,load_pred,d,source,t,co
unt)
                        if r==0:
                            if refund or legacy_plan:

grid,reference,info=plan_day(p,scene,energy,eta,kappa,hard_terminal)

original=grid.copy();obj=info['lp_objective_yuan'];grid_value=grid
                            else:
                                # 两阶段 0 时，只有存在后续发布节点的策略才能
指望 1.5p 补购。

grid,reference,Aeff,_,_,obj=plan_day_2s(p,scene,energy,eta,kappa,hard_terminal,

adjust_from=36 if active[1:] else T)
                                original=grid.copy()
                                # 价值表按场景期望购电计值。不可调整时段已由
等式强制 A^s=G，故均值在前 36 段恰等于 G，不另作拼接；不逐场景代入 A^s，否
则会与
    DP 正在决策的充放电重复计入，使反馈层误判为不会缺电而过度乐观。
                                grid_value=Aeff.mean(axis=0)
                        else:
```

```
grid,reference,obj=adjust_plan(p[t:],scene,energy,original[t:],eta,kappa,refund,hard_termina
l)
                                    grid_value=grid
                                objective.append(obj);start=t

states,values=value_table(p[t:],grid_value,scene,eta,kappa,step,hard_terminal)

active_grid[t:]=grid;ref_all[t:]=reference;source_all[t:]=source*6;decision_all[t:]=r*6
                                if keep_versions:
                                    for k in range(T-t):

versions.append((str(DATES[d].date()),r*6,source*6,t+k+1,float(original[t+k]),float(grid[k]
),

float(reference[k]),float(load_pred[d,t+k]),float(forecasts[d,source,t+k])))
                            energy,c,b,q,w=action(p[t],grid[t-
start],actual[d,0,t],actual[d,1,t],energy,states,values[t-start+1],eta)
                            results.append((initial if not results else results[-1][1],energy,c,b,q,w))
                        out=np.asarray(results)
                    inc=np.maximum(active_grid-original,0);dec=np.maximum(original-
active_grid,0)

df=pd.DataFrame(out,columns=['storage_start_kwh','storage_end_kwh','charge_kwh','disch
arge_kwh','emergency_kwh','unused_kwh'])
                    df.insert(0,'date',str(DATES[d].date()));df.insert(1,'slot',np.arange(1,T+1))
                    df['interval']=[interval(t) for t in range(T)]
                    df['price_yuan_per_kwh']=p;df['load_kwh']=actual[d,0];df['pv_kwh']=actual[d,1]
                    df['original_grid_kwh']=original;df['adjusted_grid_kwh']=active_grid

df['increase_kwh']=inc;df['decrease_kwh']=dec;df['reference_storage_kwh']=ref_all
                    df['forecast_release_hour']=source_all;df['decision_hour']=decision_all
                    df['plan_cost_yuan']=p*original;df['increase_cost_yuan']=1.5*p*inc
                    df['decrease_cost_yuan']=(-.5 if refund else .5)*p*dec
                    df['emergency_cost_yuan']=5*p*df.emergency_kwh

df['total_cost_yuan']=df[['plan_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emerg
ency_cost_yuan']].sum(axis=1)
                    df['forecast_load_kwh']=load_pred[d]
                    df['forecast_pv_kwh']=[forecasts[d,int(source_all[t]/6),t] for t in range(T)]
                    df['history_latest_date']='' if d==0 else str(DATES[d-1].date())

daily={'date':str(DATES[d].date()),'policy':policy,'initial_kwh':initial,'final_kwh':float(out[-
1,1]),
```

```
                    'emergency_slots':int((df.emergency_kwh>1e-
6).sum()),'solve_count':len(objective)}
        for col in
['original_grid_kwh','adjusted_grid_kwh','increase_kwh','decrease_kwh','emergency_kwh','
charge_kwh',

'discharge_kwh','unused_kwh','plan_cost_yuan','increase_cost_yuan','decrease_cost_yuan','
emergency_cost_yuan','total_cost_yuan']:
            daily[col]=float(df[col].sum())
        return df,daily,versions
```

附录 4：battery_core.py

介绍：问题三，用于两阶段日前购电规划、储能价值评估及实时反馈控制。

```
def predict(history, date, weeks=LOOKBACK_WEEKS):
    n = len(history)
    assert n > 0
    ids = np.arange(max(0,n-min(n,weeks*7)),n)
    same = ids[DATES[ids].dayofweek == date.dayofweek]
    if len(same)==0: same = ids
    return history[same].mean(axis=0)

def scenarios(pred, residuals, history, count=7):
    if len(residuals) >= 2:
        return np.maximum(0, pred[None,:,:]+np.asarray(residuals[-count:]))
    return history[-count:].copy()

def plan_day(p, scene, initial, eta=.9, kappa=None, hard_terminal=False):
    """共享电池参考轨迹的场景 LP。内部以 MWh 缩放，目标单位千元。"""
    if kappa is None: kappa=p[0]/eta
    s,n = scene.shape[0],len(p)
    g,c,b,e = [np.arange(k*n,(k+1)*n) for k in range(4)]
    q = np.arange(4*n,(4+s)*n).reshape(s,n)
    h=(4+s)*n; nv=h+1
    obj=np.zeros(nv);obj[g]=p;obj[q]=5*p[None,:]/s;obj[h]=kappa
    obj[c]=obj[b]=1e-8
    bounds=[(0,None)]*nv
    for t in range(n):
        bounds[c[t]]=bounds[b[t]]=(0,POWER/1000)
        bounds[e[t]]=(EMIN/1000,EMAX/1000)
    if hard_terminal:
```

```
            bounds[e[-1]]=(EREF/1000,EREF/1000)
            for t in range(n-7,n): bounds[g[t]]=(POWER/1000,None)
        eq=lil_matrix((n,nv));rhs=np.zeros(n)
        for t in range(n):
            eq[t,e[t]]=1;eq[t,c[t]]=-eta;eq[t,b[t]]=1/eta
            if t: eq[t,e[t-1]]=-1
            else: rhs[t]=initial/1000
        a=lil_matrix((s*n+n+1,nv));ub=np.zeros(s*n+n+1)
        for j in range(s):
            for t in range(n):
                row=j*n+t
                a[row,g[t]]=-1;a[row,b[t]]=-1;a[row,c[t]]=1;a[row,q[j,t]]=-1
                ub[row]=(scene[j,1,t]-scene[j,0,t])/1000
        for t in range(n):
            a[s*n+t,c[t]]=1;a[s*n+t,g[t]]=-1
            ub[s*n+t]=scene[:,1,t].min()/1000
        a[-1,e[-1]]=-1;a[-1,h]=-1;ub[-1]=-EREF/1000

res=linprog(obj,A_ub=a.tocsr(),b_ub=ub,A_eq=eq.tocsr(),b_eq=rhs,bounds=bounds,
                        method='highs',options={'primal_feasibility_tolerance':1e-
8,'dual_feasibility_tolerance':1e-8})
        if not res.success: raise RuntimeError(res.message)
        x=res.x*1000
        cancel=np.minimum(x[c],x[b]/eta**2)
        charge=np.maximum(0,x[c]-cancel); discharge=np.maximum(0,x[b]-
eta**2*cancel)
        grid=np.maximum(0,x[g]); storage=x[e]
        short=np.maximum(scene[:,0,:]+charge-grid-scene[:,1,:]-discharge,0)
        economic=float(p@grid+np.mean(short@(5*p))+kappa*max(EREF-storage[-
1],0))
        err=max(float(np.abs(np.r_[initial,storage[:-1]]+eta*charge-discharge/eta-
storage).max()),
                float(np.maximum(charge-grid-scene[:,1,:].min(axis=0),0).max()))
        assert err < 1e-4 and np.max(np.minimum(charge,discharge))<1e-5
        return grid, storage,
{'plan_objective_yuan':economic,'lp_objective_yuan':float(res.fun*1000),
                                    'lp_iterations':int(res.nit),'plan_violation_kwh':err}

    def plan_day_2s(p, scene, initial, eta=.9, kappa=None, hard_terminal=False,
adjust_from=36):
        if kappa is None: kappa=p[0]/eta
        s,n=scene.shape[0],len(p)
        g,c,b,e=[np.arange(k*n,(k+1)*n) for k in range(4)]
        base=4*n
```

```
        a=np.arange(base,base+s*n).reshape(s,n)
        q=np.arange(base+s*n,base+2*s*n).reshape(s,n)
        h=base+2*s*n;nv=h+1
        adj=np.zeros(n,bool);adj[adjust_from:]=True
        nfix=int((~adj).sum())
        obj=np.zeros(nv)
        obj[g]=p-1.5*p*adj                              # G 的净边际系数：1.5p 的
补购被抵消
        obj[a]=1.5*p[None,:]*adj[None,:]/s              # 不可调整时段 A^s 恒等于
G，该增量恒为 0
        obj[q]=5*p[None,:]/s;obj[h]=kappa
        obj[c]=obj[b]=1e-8
        bounds=[(0,None)]*nv
        for t in range(n):
            bounds[c[t]]=bounds[b[t]]=(0,POWER/1000)
            bounds[e[t]]=(EMIN/1000,EMAX/1000)
        if hard_terminal:
            bounds[e[-1]]=(EREF/1000,EREF/1000)
            for t in range(max(0,n-7),n):bounds[g[t]]=(POWER/1000,None)
        eq=lil_matrix((n+s*nfix,nv));rhs=np.zeros(n+s*nfix)
        for t in range(n):
            eq[t,e[t]]=1;eq[t,c[t]]=-eta;eq[t,b[t]]=1/eta
            if t:eq[t,e[t-1]]=-1
            else:rhs[t]=initial/1000
        for j in range(s):
            for k,t in enumerate(np.flatnonzero(~adj)):
                eq[n+j*nfix+k,a[j,t]]=1;eq[n+j*nfix+k,g[t]]=-1

aub=lil_matrix((s*n*2+s*int(adj.sum())+1,nv));ub=np.zeros(s*n*2+s*int(adj.sum())+1)
        r=0
        for j in range(s):
            for t in range(n):
                aub[r,c[t]]=1;aub[r,b[t]]=-1;aub[r,a[j,t]]=-1;aub[r,q[j,t]]=-1
                ub[r]=(scene[j,1,t]-scene[j,0,t])/1000;r+=1
                aub[r,c[t]]=1;aub[r,a[j,t]]=-1;ub[r]=scene[:,1,t].min()/1000;r+=1
                if adj[t]:aub[r,g[t]]=1;aub[r,a[j,t]]=-1;r+=1
        aub[r,e[-1]]=-1;aub[r,h]=-1;ub[r]=-EREF/1000;r+=1
        assert r==aub.shape[0]

res=linprog(obj,A_ub=aub.tocsr(),b_ub=ub,A_eq=eq.tocsr(),b_eq=rhs,bounds=bounds,
                    method='highs',options={'primal_feasibility_tolerance':1e-
8,'dual_feasibility_tolerance':1e-8})
        if not res.success:raise RuntimeError(res.message)
        x=res.x*1000
```

```
            grid=np.maximum(x[g],0);Aeff=np.maximum(x[a],0);storage=x[e]
            cancel=np.minimum(x[c],x[b]/eta**2)
            charge=np.maximum(x[c]-cancel,0);discharge=np.maximum(x[b]-
eta**2*cancel,0)
            assert (Aeff>=grid[None,:]-1e-5).all(),'A^s<G'
            assert np.max(np.abs(np.r_[initial,storage[:-1]]+eta*charge-discharge/eta-
storage))<1e-4
            assert np.max(np.minimum(charge,discharge))<1e-5
            economic=float(p@grid+np.mean(np.sum(1.5*p[None,:]*(Aeff-
grid[None,:]),axis=1))
                                    +kappa*max(EREF-storage[-1],0))
            return grid,storage,Aeff,charge,discharge,economic

      def value_table(p, grid, scene, eta=.9, kappa=None, step=100.,
hard_terminal=False):
            """共享动作的边际场景价值近似；不冒充保留全部时序相关性的精确多
阶段 DP。
            """
            if kappa is None:kappa=p[0]/eta
            g=np.asarray(grid,float)
            if g.ndim==1:g=np.broadcast_to(g,(scene.shape[0],len(g))).copy()
            assert g.shape==(scene.shape[0],len(p))
            states=np.arange(EMIN,EMAX+step/2,step)
            assert abs(states[-1]-EMAX)<1e-8
            delta=states[None,:]-states[:,None]
            charge=np.maximum(delta,0)/eta; discharge=np.maximum(-delta,0)*eta
            bus=charge-discharge
            valid=(charge<=POWER+1e-8)&(discharge<=POWER+1e-8)
            value=np.empty((len(p)+1,len(states)))
            value[-1]=kappa*np.maximum(EREF-states,0)
            if hard_terminal:value[-1]=np.where(np.abs(states-EREF)<1e-8,0,np.inf)
            for t in range(len(p)-1,-1,-1):
                  net=scene[:,0,t]-scene[:,1,t]-g[:,t]

cost=np.maximum(net[:,None,None]+bus[None,:,:],0).mean(axis=0)*(5*p[t])
                  # 仅约束普通电源可供充电；共享充电功率受最不利场景制约，与
日前模型一致。
                  allowed=valid & (charge<=g[:,t].min()+scene[:,1,t].min()+1e-8)
                  total=cost+value[t+1][None,:]
                  total[~allowed]=np.inf
                  value[t]=np.min(total,axis=1)
            return states,value

      def action(p, grid, load, pv, energy, states, future_value, eta=.9):
```

```
        """只使用本段观测，不接收当天未来实际值；储能在网格间作分段线性插值。"""
        lo=max(EMIN,energy-POWER/eta)
        hi=min(EMAX,energy+eta*min(POWER,grid+pv))
        net=load-pv-grid
        balanced=energy-net/eta if net>=0 else energy-net*eta
        candidates=np.unique(np.r_[states[(states>=lo)&(states<=hi)],lo,hi,
                                   np.clip(energy,lo,hi),np.clip(balanced,lo,hi)])
        change=candidates-energy
        c=np.maximum(change,0)/eta;b=np.maximum(-change,0)*eta
        emergency=np.maximum(net+c-b,0)
        values=5*p*emergency+np.interp(candidates,states,future_value)
        finite=np.isfinite(values)
        if not finite.any():raise RuntimeError('No feasible real-time action')
        best=values[finite].min()
        # 数值等价时最小吞吐量；不把实际吞吐惩罚计入费用。
        ids=np.flatnonzero(finite & (values<=best+1e-7))
        j=ids[np.argmin((c+b)[ids])]
        unused=max(grid+pv+b[j]+emergency[j]-load-c[j],0)
        return
float(candidates[j]),float(c[j]),float(b[j]),float(emergency[j]),float(unused)
```

附录 5： core.py

介绍：问题四，实现负载与电价预测、联合场景生成以及浮动电价下的滚动购电与储能优化。

```
def predictions(data):
    """Each day / release built exclusively from information available then."""
    base=np.zeros_like(data); prices=np.full((365,4,T),np.nan)
    for d in range(1,365):
        ids=np.arange(max(0,d-56),d);
same=ids[DATES[ids].dayofweek==DATES[d].dayofweek]
        base[d,:2]=data[same if len(same) else ids,:2].mean(axis=0)
        base[d,2]=data[max(0,d-28):d,2].mean(axis=0)
        for r in range(4):
            off=r*36
            correction=float((data[d,2,max(0,off-6):off]-base[d,2,max(0,off-
6):off]).mean()) if off else 0.
            prices[d,r,off:]=np.maximum(.001,base[d,2,off:]+correction*np.exp(-
np.arange(T-off)/36))
    return base,prices
```

```
    def make_scene(data,fc,base,prices,d,r,source,count=7):
        off=r*36;ids=np.arange(max(1,d-count),d)
        point=np.stack([base[d,0,off:],base[d,1,off:] if source is None else
fc[d,source,off:],prices[d,r,off:]])
        if not len(ids):scene=point[None].copy()
        else:
            past=base[ids,:,off:].copy();past[:,2]=prices[ids,r,off:]
            if source is not None:past[:,1]=fc[ids,source,off:]
            scene=point+data[ids,:,off:]-past
        scene[:,:2]=np.maximum(scene[:,:2],0);scene[:,2]=np.maximum(scene[:,2],.001)
        assert np.isfinite(scene).all()
        return scene,point

    def
plan(scene,initial,kappa,original=None,eta=.9,hard=False,adj_from=None,adj_coef=SHOR
T_ADJ):
        """0 时共享承诺 LP；original 给出时是节点调整 LP（A>=G0，增购按
1.5p）。
        adj_from 把"节点后可调整"的时段缺额从 5p 应急改按 adj_coef 的增购价计
价。0 时若一律按 5p 给缺额定阶，等价于把承诺抬到 80%分位，而合同下增购只需
1.5p、最优分位是 1/1.5≈67%，于是系统性多承诺。缺额系数与增购成本必须同价，
否则 0 时计划与节点实际面临的价格不一致。adj_from=None 表示全天不可调整
（Q4-2 及 only0 对照）。
        """
        s,_,n=scene.shape; price=scene[:,2]; p=price.mean(0)
        g,c,b,e=[np.arange(k*n,(k+1)*n) for k in range(4)]
        q=np.arange(4*n,(4+s)*n).reshape(s,n);h=(4+s)*n;nv=h+1
        coef=np.full(n,5.) if adj_from is None else
np.where(np.arange(n)>=adj_from,adj_coef,5.)
        obj=np.zeros(nv);obj[g]=(1.5 if original is not None else 1.)*p
        obj[q]=coef[None,:]*price/s;obj[h]=kappa;obj[c]=obj[b]=1e-8
        bounds=[(0,None)]*nv
        for t in range(n):

bounds[c[t]]=bounds[b[t]]=(0,POWER/1000);bounds[e[t]]=(EMIN/1000,EMAX/1000)
            if original is not None:bounds[g[t]]=(original[t]/1000,None)
        if hard:
            bounds[e[-1]]=(6.,6.)
            for t in range(max(0,n-
7),n):bounds[g[t]]=(max(POWER/1000,bounds[g[t]][0]),None)
        eq=lil_matrix((n,nv));rhs=np.zeros(n)
        for t in range(n):
            eq[t,e[t]]=1;eq[t,c[t]]=-eta;eq[t,b[t]]=1/eta
```

```
                if t:eq[t,e[t-1]]=-1
                else:rhs[t]=initial/1000
            mat=lil_matrix((s*n+n+1,nv));ub=np.zeros(s*n+n+1)
            for j in range(s):
                for t in range(n):
                    i=j*n+t;mat[i,g[t]]=-1;mat[i,b[t]]=-1;mat[i,c[t]]=1;mat[i,q[j,t]]=-1
                    ub[i]=(scene[j,1,t]-scene[j,0,t])/1000
            for t in range(n):
                mat[s*n+t,c[t]]=1;mat[s*n+t,g[t]]=-1;ub[s*n+t]=scene[:,1,t].min()/1000
            mat[-1,e[-1]]=-1;mat[-1,h]=-1;ub[-1]=-6.

res=linprog(obj,A_ub=mat.tocsr(),b_ub=ub,A_eq=eq.tocsr(),b_eq=rhs,bounds=bounds,met
hod='highs',
                            options={'primal_feasibility_tolerance':1e-
8,'dual_feasibility_tolerance':1e-8})
            if not res.success:raise RuntimeError(res.message)
            x=res.x*1000;delta=np.minimum(x[c],x[b]/eta**2)
            ch=np.maximum(0,x[c]-delta);dis=np.maximum(0,x[b]-
eta**2*delta);grid=np.maximum(x[g],0)
            short=np.maximum(scene[:,0]+ch-grid-scene[:,1]-dis,0)
            economic=float(p@grid*(1.5 if original is not None else
1.)+(coef[None,:]*price*short).sum()/s+kappa*max(6000-x[e[-1]],0))
            if original is not None:economic-=float(1.5*p@original)
            assert np.max(abs(np.r_[initial,x[e][:-1]]+eta*ch-dis/eta-x[e]))<1e-4
            return grid,x[e],economic

        def value_table(scene,grid,kappa,eta=.9,step=100.,hard=False):
            states=np.arange(EMIN,EMAX+step/2,step);n=len(grid)
            delta=states[None,:]-states[:,None];c=np.maximum(delta,0)/eta;b=np.maximum(-
delta,0)*eta
            valid=(c<=POWER+1e-8)&(b<=POWER+1e-8);bus=c-b
            values=np.empty((n+1,len(states)));values[-1]=kappa*np.maximum(6000-
states,0)
            if hard:values[-1]=np.where(abs(states-6000)<1e-8,0,np.inf)
            for t in range(n-1,-1,-1):
                net=scene[:,0,t]-scene[:,1,t]-grid[t]

cost=(np.maximum(net[:,None,None]+bus,0)*5*scene[:,2,t,None,None]).mean(0)
                allowed=valid&(c<=grid[t]+scene[:,1,t].min()+1e-8)
                costs=cost+values[t+1];costs[~allowed]=np.inf;values[t]=costs.min(axis=1)
            return states,values

        def action(price,grid,load,pv,energy,states,future,eta=.9):
```

```
            lo=max(EMIN,energy-
POWER/eta);hi=min(EMAX,energy+eta*min(POWER,grid+pv));net=load-pv-grid
            balanced=energy-net/eta if net>=0 else energy-net*eta

candidates=np.unique(np.r_[states[(states>=lo)&(states<=hi)],lo,hi,np.clip(energy,lo,hi),np.
clip(balanced,lo,hi)])
            change=candidates-energy;c=np.maximum(change,0)/eta;b=np.maximum(-
change,0)*eta
            emergency=np.maximum(net+c-
b,0);cost=5*price*emergency+np.interp(candidates,states,future)
            finite=np.isfinite(cost)
            if not finite.any():raise RuntimeError('No feasible real-time action')
            ids=np.flatnonzero(finite&(cost<=cost[finite].min()+1e-
7));j=ids[np.argmin((c+b)[ids])]
            return candidates[j],c[j],b[j],emergency[j],max(grid+pv+b[j]+emergency[j]-load-
c[j],0)

      def
simulate_day(data,fc,base,prices,d,initial,branch=2,policy='all',eta=.9,step=100.,hard=False
,short_adj=SHORT_ADJ):

energy=initial;original=np.zeros(T);active=np.zeros(T);reference=np.full(T,initial)
            point_used=np.zeros((3,T));decision=np.zeros(T,int);source_used=np.full(T,-
1,int);versions=[];rows=[]
            if d==0:
                  for t in range(T):rows.append([energy,energy,0.,0.,max(data[d,0,t]-
data[d,1,t],0),max(data[d,1,t]-data[d,0,t],0)])
                  basegrid=np.zeros(T)
            else:
                  kappa=base[d,2,0]/eta
                  # 分支 3 首个调整节点在 t=36；only0 对照不做节点调整，仍按全天
5p
                  adjustable=36 if (branch==3 and policy!='only0') else None
                  for t in range(T):
                        if t%36==0:
                              r=t//36
                              source=None if branch==2 else (0 if policy in ('only0','stale_all')
else (2 if policy=='stale18' and r==3 else r))
                              scene,point=make_scene(data,fc,base,prices,d,r,source)
                              if t==0:

grid,ref,obj=plan(scene,energy,kappa,eta=eta,hard=hard,adj_from=adjustable,adj_coef=sho
rt_adj)
```

```
original=grid.copy();active[:]=grid;reference[:]=ref;basegrid=baseline(scene)
                            elif branch==3 and policy!='only0':
                                grid,ref,obj=plan(scene,energy,kappa,original[t:],eta,hard)
                                active[t:]=grid;reference[t:]=ref
                            states,values=value_table(scene,active[t:],kappa,eta,step,hard)
                            point_used[:,t:]=point;start=t;source_used[t:]=-1 if source is None
else source*6
                            if t==0 or (branch==3 and policy!='only0'):
                                decision[t:]=r*6
                                for k in
range(t,T):versions.append([str(DATES[d].date()),r*6,-1 if source is None else
source*6,k+1,original[k],active[k]])
                        old=energy

energy,c,b,e,w=action(data[d,2,t],active[t],data[d,0,t],data[d,1,t],energy,states,values[t-
start+1],eta)
                        rows.append([old,energy,c,b,e,w])

df=pd.DataFrame(rows,columns=['storage_start_kwh','storage_end_kwh','charge_kwh','disc
harge_kwh','emergency_kwh','unused_kwh'])

df.insert(0,'date',str(DATES[d].date()));df.insert(1,'slot',np.arange(1,145));df['interval']=[int
erval(t) for t in range(T)]
            for i,col in
enumerate(['load_kwh','pv_kwh','price_yuan_per_kwh']):df[col]=data[d,i]
            for i,col in
enumerate(['forecast_load_kwh','forecast_pv_kwh','forecast_price']):df[col]=point_used[i]

df['original_grid_kwh']=original;df['adjusted_grid_kwh']=active;df['reference_storage_kwh
']=reference
            df['decision_hour']=decision;df['forecast_release_hour']=source_used
            df['increase_kwh']=np.maximum(active-
original,0);df['decrease_kwh']=np.maximum(original-active,0)

p=data[d,2];df['plan_cost_yuan']=p*original;df['increase_cost_yuan']=1.5*p*df.increase_k
wh

df['decrease_cost_yuan']=.5*p*df.decrease_kwh;df['emergency_cost_yuan']=5*p*df.emerg
ency_kwh

df['total_cost_yuan']=df[['plan_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emerg
ency_cost_yuan']].sum(axis=1)
```

```
            df['baseline_cost_yuan']=p*(basegrid+5*np.maximum(data[d,0]-data[d,1]-
basegrid,0))
            df['history_latest_date']='' if d==0 else str(DATES[d-1].date())

daily={'date':str(DATES[d].date()),'initial_kwh':initial,'final_kwh':energy,'branch':branch,'p
olicy':policy}
            for col in
['original_grid_kwh','adjusted_grid_kwh','increase_kwh','decrease_kwh','emergency_kwh','
charge_kwh','discharge_kwh','unused_kwh','plan_cost_yuan','increase_cost_yuan','decrease
_cost_yuan','emergency_cost_yuan','total_cost_yuan','baseline_cost_yuan']:
                daily[col]=float(df[col].sum())
            daily['emergency_slots']=int((df.emergency_kwh>1e-6).sum())
            return df,daily,versions
```